Two-stage scheduling of dual-source mixed-flow rotating-chamber cluster tools under full-occupancy processing

Lizhen Shao[a,*], Lincong Wei[a], Wei Wang[a]

[a]*Department of Control Science and Engineering, University of Science and Technology Beijing, No. 30 Xueyuan Road, Haidian District, Beijing 100083, China*

Abstract

Advanced manufacturing processes place increasing demands on the throughput, space utilization, and operational stability of thin-film deposition systems, motivating the development of cluster tools with multi-position rotating chambers. Difficult-to-remove residues from carbon-containing film processes expose unoccupied positions to contamination during processing and cleaning, requiring auxiliary wafers to fill those positions. Auxiliary and product wafers are stored in different regions and form mixed flows through shared robots and multi-position rotating chambers. We develop a two-stage mixed-integer linear programming model. Stage one reduces product-batch makespan subject to residency limits and other physical constraints; stage two preserves its upper bound and principal structure while reducing waiting and variation in consecutive production-cycle intervals. The solution approach combines structured initialization, time-limited MILP search, and timing refinement, and evaluates the role of learned cut selection. In the historical fixed-layout benchmark, the optimal full-occupancy process modes for a 16-product lot are pure 4×1, mixed, and pure 2×2 operation at cleaning thresholds of 3, 4, and 6, respectively. Cold starts for four configurations with three seeds each obtain no feasible solution within 300 s, whereas main solves with precomputed initial solutions all yield valid schedules. Equal-budget refinement over eight configurations preserves product makespan and reduces total waiting by 22.29%–50.25%. The study provides a common modeling and evaluation basis for mode comparison and feasible scheduling within limited computation budgets for rotating-chamber cluster tools under full-occupancy processing.

Keywords: semiconductor manufacturing systems, full-occupancy processing, dummy-wafer scheduling, two-stage optimization, mixed-integer linear programming, learning-enhanced exact optimization

1. Introduction

Plasma-enhanced chemical vapor deposition (PECVD) is used in semiconductor thin-film fabrication. Cluster tools (CTs) connect process modules, robots, and load locks into a continuous wafer-handling route. Processing and transport times, load-lock switching, cleaning, and residency constraints jointly determine feasible operation and equipment output [1]. Three-dimensional device structures and multilayer integration increase demands on deposition uniformity and processing capability [2]. Multi-space process modules and multi-finger robots provide a structural basis for parallel processing within a tool [3, 4].

Process-quality prediction provides a related but distinct context for equipment scheduling. Maitra et al. review virtual metrology methods that predict product quality from equipment state, process variables, and sampled physical measurements [5]. For amorphous carbon deposition, Choi and Hong use equipment-state data and in situ optical emission spectroscopy in neural-network film-thickness prediction [6]. Here, recipes and mode eligibility that satisfy product-quality requirements are inputs to resource scheduling; schedule quality does not replace film-thickness prediction or recipe qualification.

This study considers a PECVD CT with four-position rotating chambers. The operating decision starts from the full-occupancy requirement of the studied recipes: all four positions must contain wafers during production, with dummy wafers filling positions not occupied by products. We use auxiliary wafer to denote a reusable dummy wafer used for production filling or cleaning occupancy. Dummies do not contribute to product output, but their handling, rotation, transport, and repeated loans consume resources. Products enter from atmospheric load ports through alignment and load locks; in dual-source operation, auxiliaries come from vacuum storage. These source and route differences form part of the equipment conditions for mode comparison.

Full occupancy couples product supply with auxiliary reuse. Auxiliary wafers fill positions when products cannot occupy the entire chamber. Cleaning occupies auxiliary wafers and chambers and changes their subsequent availability. Product arrival therefore does not alone permit processing: auxiliary availability, robot service order, and cleaning boundaries must also be satisfied. The two wafer routes in Fig. 1 require coordinated scheduling.

Two prescribed modes are considered: 4×1 parallel processing gives four wafers one synchronous exposure, whereas 2×2 chained processing uses paired transfers and rotation to give each product two consecutive exposures. Recipes and chamber

*Corresponding author.
Email address: lshao@ustb.edu.cn (Lizhen Shao)

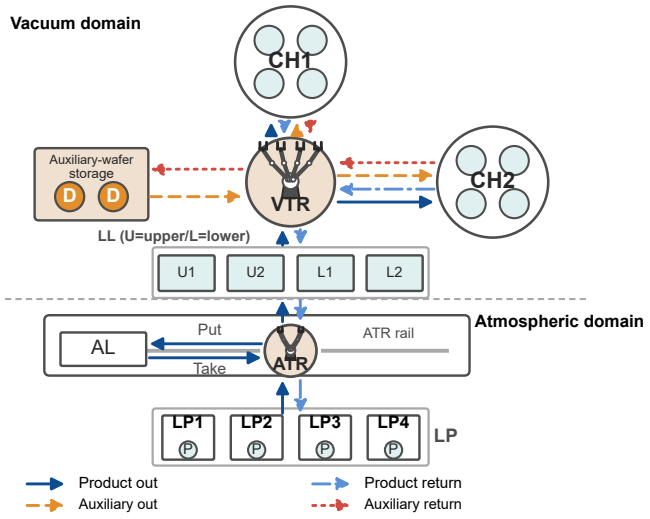


Figure 1: Dual-source mixed-flow rotating-chamber CT topology and wafer routes.

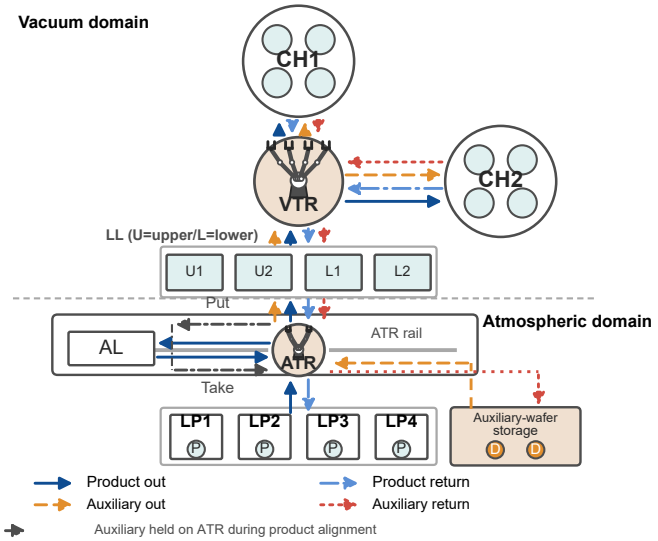


Figure 2: Single-source equipment topology and wafer routes. Auxiliary storage is beside LP4. Products undergo AL alignment. A separately supplied auxiliary moves directly from ATR to the upper load lock; when collected with a product, it remains on ATR during the AL visit and enters the lock with the aligned product. Both types reach the chambers through VTR and return through the lower load lock and ATR.

operations define these modes. Dummy wafers required for full occupancy give them different resource costs for partial batches, chain initialization and termination, and cleaning transitions. Equal cumulative nominal processing time per product therefore need not yield equal batch makespan. Mode preference must be assessed against lot size, cleaning threshold, dummy supply, and handling conditions. Product modes are fixed inputs within each solve. Mode selection is supported by separately constructing pure and mixed configurations and comparing validated schedules and bounds, rather than freely changing product recipes within one solve.

CT studies established analytical relationships between processing, handling, and throughput [7] and used linear programming, heuristics, and timed Petri nets to obtain schedules under residency constraints [8–10]. Later models address revisiting processes, multiple wafer types, and selectable modules [11–13]. Clean/dirty-hand separation and steady-state analysis of four-finger robots and four-space modules, together with joint optimization of two-type wafer releases and handling, supports more complex configurations [4, 14]. Finite batches additionally involve chain initialization, termination, and partial-batch filling, requiring coordinated product and auxiliary handling.

Periodic-cleaning scheduling uses task-sequence enumeration, partial loading and timing control, virtual-wafer occupancy, and chamber-state-dependent triggers to coordinate maintenance and production [15–18]. Shared-buffer synchronization and dynamic action screening address deadlock during failure shutdown and noncyclic operation [19, 20]; waiting constraints also affect equipment output and upstream control [21]. Here, cleaning simultaneously changes chain segmentation, auxiliary occupancy, and vacuum-robot order. Scheduling product processing or cleaning independently would omit these dependencies.

Timing quality requires distinguishing batch efficiency from coordination between operations. JMS research links waiting limits to effective output [21], while JIT scheduling seeks completion times aligned with downstream acceptance [22]. For the present tool, reducing avoidable waiting and variation in repeated production-cycle intervals can support predictable supply and pickup plans. However, timely demand-driven coordination, uniform wafer residency, and regular cycle-start intervals are distinct criteria. This study directly optimizes waiting and cycle cadence and imposes hard residency limits; these measures do not establish equal residency or strict JIT control.

Equipment-state studies provide an engineering basis for parameter updates. Sun et al. report differences in processing time across chambers and use equipment-data analysis to locate steps responsible for throughput loss [23]. Chen et al. detect sensory-data anomalies in an HDP-CVD tool using recipe-cycle extraction, time and spectral features, and unsupervised autoencoders [24]. Biebl et al. connect time-to-failure interval prediction and Bayesian root-cause analysis to maintenance recommendations and planning [25]. These studies support state assessment and maintenance decisions; the present work coordinates production, handling, and cleaning for given equipment parameters. Translating state estimates into scheduling parameters and triggering rescheduling require a separate interface and validation.

Two hardware-deviation mechanisms motivate schedule updates: actual action durations can deviate from nominal values, and chambers or robots can differ in performance. Either can change resource availability and require mode evaluation and scheduling to be recomputed for updated parameters. JMS research on uncertain durations explicitly distinguishes proactive scheduling from reactive adjustment [26]. Here, the computational task is time-limited scheduling for a given parameter snapshot: obtain a complete constraint-checked schedule within the budget, then improve makespan and timing qual-

ity. Current experiments do not inject online stochastic deviations or implement measurement feedback and closed-loop rescheduling; they validate scheduling only for the tested deterministic configurations.

These wafer and resource constraints define a mixed-integer scheduling problem. Learning can select CT actions directly or assist mathematical optimization. Reinforcement learning with look-ahead search supports noncyclic CT control [27]; reinforcement-learning selection, cut ranking, and look-ahead imitation select candidate inequalities in MILP [28–30]. HEM jointly determines cut counts and ordered sequences [31]. We retain this hierarchical decomposition and use equipment-variable structure for cut selection while representing process feasibility in the MILP.

The model and solution approach address which process mode to use under specified operating conditions and how to obtain a valid schedule in time. The contributions are:

1. A joint model of product and dummy resource occupancy connects process modes to batch performance through wafer pairing, auxiliary loans, chamber assignment, complete production chains, and cleaning boundaries, supporting comparisons of pure and mixed configurations under matched equipment conditions.

2. A makespan-first two-stage MILP reduces product-batch completion time within residency limits, then reduces waiting and cycle-interval variation while preserving the makespan bound and principal structure. Operating efficiency, timing quality, and residency uniformity are distinguished explicitly.

3. A solution workflow combines SPBS initialization, time-limited MILP search, and timing refinement. Feasible-schedule availability, product makespan, waiting, and bound quality separately measure its performance, with matched-budget comparisons testing the incremental contribution of learned cut selection.

Section 2 defines wafer routes, process constraints, and the two-stage model. Section 3 presents initialization and cut selection. Section 4 analyzes mode choice, schedule quality, and computational performance. Section 5 concludes.

2. Problem description and two-stage scheduling model

All material considered here consists of wafers: product wafers and auxiliary wafers. Products receive their prescribed recipe; auxiliary wafers fill production positions or occupy the chamber during cleaning and are reused after return. A chamber operation is either production or cleaning. Production means recipe processing in the 4×1 or 2×2 mode; cleaning uses only auxiliary wafers. One production or cleaning operation constitutes one chamber cycle. A production cycle specifically denotes a recipe-processing cycle and excludes cleaning. Loading, unloading, and rotation are separate actions; waiting is the interval between consecutive actions.

Two decision levels are distinguished. Mode comparison constructs pure 4×1, pure 2×2, and prescribed mixtures once product-recipe requirements are qualified, holding common conditions such as lot size, source routes, dummy inventory, cleaning rule, and equipment parameters fixed. Scheduling within each configuration jointly determines wafer pairing, chamber assignment, chamber cycles, shared-resource order, and action times for its prescribed recipes. Whether the alternative recipes meet the same product-quality requirements is an input qualification condition; scheduling results do not establish recipe equivalence.

Mode-selection criterion.. Each candidate configuration must satisfy complete routes, full occupancy, resource capacities, and residency limits before its schedule is compared under matched computational settings. A smaller feasible makespan identifies a better schedule obtained within the current budget. Establishing mode dominance additionally requires its feasible upper bound to lie below all competing valid lower bounds, or comparison of proven optima. Overlapping bounds leave the ranking unresolved; timeout without an incumbent does not establish physical infeasibility. For similar product makespans, waiting, cadence, and auxiliary-occupancy characteristics further describe operating costs without changing the optimization priorities used here.

Inputs comprise action durations, resource capacities, residency limits, cleaning threshold, and auxiliary inventory. Reducing batch completion time means reducing the last product-return time $C_{\max}$, rather than changing recipe-prescribed processing durations or minimizing the sum of action durations. Stage one prioritizes $C_{\max}$; stage two improves waiting and production cadence while preserving its bound. Stable coordination is therefore an engineering aim of timing refinement, while equal residency or fixed cadence is not a prerequisite enforced before stage one in the current model. A feasible schedule specifies wafers, resources, and event times for checking precedence, conflicts, and operating metrics.

The cleaning threshold satisfies $H \geq 2$. The baseline auxiliary inventory is sized at four wafers per four-position rotating chamber, giving eight wafers for the two-chamber tool. All chambers share this inventory; individual wafers are not dedicated to a chamber. This total covers simultaneous auxiliary occupancy of every chamber position, while availability at a particular time depends on ongoing loans and returns. Separately reported inventory sensitivity cases vary this baseline. Action durations are deterministic within each solve. No buffers are assumed beyond LPs, aligners, load locks, rotating chambers, and vacuum storage.

Table 1 summarizes sets, inputs, and capacity bounds; Table 2 lists core decisions, derived indicators, and linearization auxiliaries. All events use start and end times s_o, e_o. For a fixed-duration physical action, $e_o = s_o + d_o b_o$ can be substituted directly. For a resource-holding interval spanning several actions, the endpoints are linked to the corresponding pickup and placement events, and its duration is not prescribed. Loading, unloading, rotation, cycle, and cleaning-stage times are indexed aliases of these physical events, not independent clocks.

Subscripts w, i, p, c, q denote a product wafer, wafer-route task, PW pair, chamber, and candidate chamber cycle, respectively; o, o' denote actions or resource-occupancy tasks, and α denotes a waiting item. In a_{wpr}, d_{pr}, $r = 0, 1$ identifies a member position within a pair; in $y^r_{oo'}$, r identifies a resource. The index $\ell \in \Gamma_o$ identifies a physical hand or slot. A loan request $j = (p, r)$ identifies one use of an auxiliary wafer, whereas τ identifies a reusable physical wafer. Cleaning segment e and production task v are derived from the cycle sequence and used to describe cleaning windows.

The labels ζ, ψ, cl denote the two production recipes and cleaning. The labels load, unload, rot±, ref, return, and max denote loading, unloading, rotation before/after a cycle, a reference stage, product return, and a maximum. They are semantic labels rather than additional decision indices. All times are in seconds. A binary indicator is 1 when its stated choice holds and 0 otherwise. Derived quantities abbreviate expressions used in several constraints; their defining relationships introduce no independent decision freedom.

2.1. Single- and dual-source physical routes

Let $\mathcal{W}$ be the product-wafer set and $\mathcal{I}$ the set of wafer-route tasks. Each product has one complete route; each loan of an auxiliary wafer has one supply-and-return route. One physical auxiliary wafer can serve several successive loan tasks.

Let $\mathcal{S} = \{A, V\}$ denote atmospheric and vacuum source regions. The source configuration is $\sigma \in \{0, 1\}$ for single- and dual-source supply, respectively. LP1–LP4 specify product locations within the atmospheric region; each product returns to its original LP. Products satisfy $\chi_{wA} = 1, \chi_{wV} = 0$. Auxiliary task i satisfies $\chi_{iA} = 1 - \sigma, \chi_{iV} = \sigma$. Source-membership parameters obey

$$\sum_{s \in \mathcal{S}} \chi_{is} = 1, \qquad i \in \mathcal{I}. \tag{1}$$

Equation (1) assigns a source region to every route task. Under single-source supply, products occupy LP1–LP4 and auxiliary wafers are stored beside LP4 in the atmospheric region. Both share ATR and load-lock interfaces, but only products require AL alignment. Under dual-source supply, products still originate at LP1–LP4, whereas auxiliary wafers originate in vacuum storage. Source configuration concerns the regions supplying the two wafer types, not the number of LPs.

In both configurations, products follow LP–ATR–AL–ATR–upper load lock–VTR–process chamber–VTR–lower load lock–ATR–original LP. Alignment precedes the load-lock pressure transition and execution of the prescribed recipe in the chamber.

Single-source auxiliary wafers travel from storage beside LP4 through ATR, the upper load lock, and VTR to the chamber, and return through VTR, the lower load lock, and ATR. A separately supplied auxiliary wafer bypasses AL. During an accompanying entry with a product, it remains on ATR while the product is aligned, and both enter the lock after product retrieval. Neither route aligns the auxiliary wafer.

Dual-source auxiliary wafers travel from vacuum storage through VTR to the chamber and return through VTR after use. Figures 2 and 1 show both configurations. These routes specify the required physical stages; paired-transfer choices and their timing conditions are given in Section 2.3.

2.2. Assignments of wafers, PW pairs, chambers, and cycles

Individual wafers are first organized into PW pairs, which are assigned to compatible chambers and chamber cycles. Production and cleaning use the same candidate-cycle set. Exposure counts and cleaning triggers are specified in Section 2.4.

Wafer pairing and chamber selection. A wafer pair (PW pair) contains two products, one product and one auxiliary wafer, or two auxiliary wafers. Product LP ℓ_w and recipe m_w are given; same-recipe products may be paired across LPs. The model chooses the members of each candidate pair $p \in \mathcal{P}$. Binary a_{wpr} assigns product w to member position $r \in \{0, 1\}$, d_{pr} indicates auxiliary occupancy, and v_p activates the pair:

$$\begin{aligned} \sum_{p,r} a_{wpr} &= 1 \qquad (w \in \mathcal{W}), \\ \sum_{w} a_{wpr} + d_{pr} &= v_p \qquad (p \in \mathcal{P},\ r = 0, 1). \end{aligned} \tag{2}$$

Equation (2) assigns every product to exactly one pair and fills both positions of every active pair.

Let $\mathcal{M} = \{\zeta, \psi, \mathrm{cl}\}$ denote 4×1 production, 2×2 production, and cleaning. The set $C(m)$ contains chambers compatible with operation type m. Binary variables μ_{pm} and z_{pc} choose the pair's operation type and chamber and prohibit incompatible combinations:

$$\begin{aligned} \sum_{m} \mu_{pm} &= v_p, \qquad \sum_{c} z_{pc} = v_p, \\ a_{wpr} &\le \mu_{p,m_w}, \\ z_{pc} + \mu_{pm} &\le 1 \qquad (c \notin C(m)). \end{aligned} \tag{3}$$

Product recipe m_w and source configuration σ are inputs; pairing, chamber assignment, and operation order are decisions. Binary B_p identifies a product-containing pair and B^{ψ}_p a pair containing 2×2 products:

$$\begin{aligned} B_p \le \sum_{w,r} a_{wpr} \le 2B_p, &\qquad B_p + \mu_{p,\mathrm{cl}} \le 1, \\ B^{\psi}_p \le B_p, \quad B^{\psi}_p \le \mu_{p\psi}, &\qquad B^{\psi}_p \ge B_p + \mu_{p\psi} - 1. \end{aligned} \tag{4}$$

Candidate-capacity bounds. Auxiliary-only pairs provide filling or cleaning; product-containing pairs cannot perform cleaning. Let $N = |\mathcal{W}| = N_\zeta + N_\psi$ be the number of products, excluding reusable auxiliary wafers. Here N_ζ, N_ψ count products of the two recipes. The following conservative capacities reserve enough indices for joint pairing and cycle decisions:

$$\begin{aligned} T &= N_\zeta + 2N_\psi, \qquad J_{\max} = T, \\ P_{\max} &= N + T + 2J_{\max}, \qquad Q_{\max} = T + J_{\max}. \end{aligned} \tag{5}$$

The bound T limits the total number of production cycles across the tool. Each 4×1 product receives one exposure and each 2×2 product receives two; every production cycle contains at least one product. Thus the total number of production cycles

Table 1: Core sets and input parameters

Symbol	Type	Definition and role
$\mathcal{W}, \mathcal{I}$	Sets	$\mathcal{W}$: products, indexed by w; $\mathcal{I}$: wafer-route tasks, indexed by i, including each product route and each auxiliary-wafer loan route.
$\mathcal{S}, \sigma, \chi_{is}$	Set/binary inputs	$\mathcal{S} = \{A, V\}$: source regions, indexed by s (atmospheric/vacuum). $\sigma = 0/1$: single/dual source. $\chi_{is} = 1$: task i originates in region s.
$\ell_w, m_w, C(m)$	Inputs/set	ℓ_w: LP of product w; $m_w \in \{\zeta, \psi\}$: its prescribed recipe; $C(m)$: chambers compatible with operation type m.
N	Integer input	Number of product wafers, $N = \lvert\mathcal{W}\rvert = N_\zeta + N_\psi$; excludes auxiliary wafers.
$\mathcal{P}, \mathcal{M}$	Sets	$\mathcal{P}$: candidate PW pairs, indexed by p; $\mathcal{M} = \{\zeta, \psi, \mathrm{cl}\}$: operation types, indexed by m, denoting 4×1 production, 2×2 production, and cleaning.
C	Set	$C = \{\mathrm{CH1}, \mathrm{CH2}\}$: process chambers, indexed by c.
$\mathcal{Q}_c$	Index set	$\mathcal{Q}_c = \{1, \ldots, Q_{\max}\}$: candidate chamber cycles in chamber c, indexed by q.
N_ζ, N_ψ; $T, J_{\max}$	Counts/bounds	N_ζ, N_ψ: product counts by recipe (recipe-label subscripts). T: upper bound on total production cycles; $J_{\max}$: upper bound on total cleanings across the tool.
$P_{\max}, Q_{\max}$	Capacity bounds	$P_{\max}$: candidate-pair capacity for the entire lot. $Q_{\max}$: tool-wide bound on production-plus-cleaning cycles, also used as each chamber's candidate capacity. The label max denotes an upper bound.
H	Integer input	Maximum production count in one chamber since its initially clean state or latest cleaning. Cleaning is optional at a valid boundary after 1 to $H-1$ cycles and mandatory at H.
$O, \mathcal{A}$	Sets	O: candidate physical actions, indexed by o, o'; $\mathcal{A}$: precedence arcs, where (o, o') requires o before o'.
$\mathcal{A}^{\mathrm{int}}, \mathcal{A}_{is}(\gamma)$	Conditional arc sets	$\mathcal{A}^{\mathrm{int}}$: common chamber-internal arcs (int is a label); $\mathcal{A}_{is}(\gamma)$: supply/return arcs for task i from region s, activated by transfer choices γ.
d_o, r_c	Time inputs	d_o: fixed duration of action o; r_c: half-turn duration of chamber c, both in seconds.
$\delta_{oo'}, \rho^r_{oo'}, \rho^{\mathrm{VTR}}_{cc'}$	Time inputs	$\delta_{oo'}$: minimum precedence gap; $\rho^r_{oo'}$: repositioning/reset time on resource r between tasks o, o'; $\rho^{\mathrm{VTR}}_{cc'}$: VTR travel time from chamber c to c'.
$M, \overline{R}_{wr}$	Time bounds	M: valid large constant for conditional timing constraints; $\overline{R}_{wr}$: maximum residency of wafer w in constrained equipment or route region r, in seconds.
$\overline{T}, d_m$	Time inputs	$\overline{T}$: valid schedule-time horizon; d_m: fixed duration of operation type m, in seconds. An overbar denotes an upper bound.
$\mathcal{E}_c, \mathcal{V}_c$	Sets	$\mathcal{E}_c$: cleaning segments of chamber c, indexed by e; $\mathcal{V}_c$: its production tasks, indexed by v, each a 4×1 batch or 2×2 production cycle.
$\mathcal{J}^D_c, \mathcal{T}_c$	Sets	$\mathcal{J}^D_c$: auxiliary-loan requests assigned to chamber c; $j = (p, r)$ identifies pair p and member position r (D labels auxiliary demand). $\mathcal{T}_c$: accessible physical auxiliary identities, indexed by τ.
$\mathcal{R}$	Set	$\mathcal{R}$: shared physical resources, indexed by r, including robot actuators, hands, module slots, chambers, and physical auxiliary wafers.
Γ_o	Allowed-unit set	Γ_o: physical units eligible for occupancy task o, indexed by ℓ. VTR units D_1, D_2 are dirty hands and C_1, C_2 clean hands; digits distinguish hands of one class.
$\mathrm{in}(w, r), \mathrm{out}(w, r)$	Event maps	$\mathrm{in}(w, r), \mathrm{out}(w, r)$: action indices for wafer w entering/leaving constrained region r, used to reference event times.
$\mathcal{D}_{\mathrm{fix}}, \mathcal{Q}^w, \mathcal{I}_h$	Index sets	$\mathcal{D}_{\mathrm{fix}}$: fixed second-stage components, indexed by d; $\mathcal{Q}^w$: waiting items, indexed by α (w labels waiting); $\mathcal{I}_h$: central indices (c, q) of three consecutive production cycles in one chamber and 2×2 chain.
$\lambda_1, \ldots, \lambda_4$	Nonnegative weights	$\lambda_1, \lambda_2, \lambda_3, \lambda_4$: weights on total waiting, maximum individual waiting, cadence variation, and soft-limit excess, respectively. Numeric subscripts identify objective terms.
$\bar{\omega}_\alpha, \varepsilon_C$	Time inputs	$\bar{\omega}_\alpha$: soft upper limit of waiting item α; ε_C: nonnegative second-stage makespan tolerance (C labels completion time), in seconds.

is at most $N_\zeta + 2N_\psi$. The threshold H limits the production count in each chamber since its initially clean state or latest cleaning. Cleaning is optional at a valid boundary after 1 to $H-1$ production cycles and mandatory when the count reaches H. Initial empty cleaning and consecutive cleanings are prohibited, so each cleaning is preceded by at least one production cycle. Hence $J_{\max} = T$ safely bounds total cleanings. This bound retains all valid early-cleaning choices; it does not require T cleanings.

The capacity $P_{\max}$ bounds the candidate pair set $\mathcal{P} = \{1, \ldots, P_{\max}\}$ for the entire lot. There are at most N product-containing pairs because each contains a product and every product belongs to exactly one pair. Each production cycle contains two PW pairs and at least one product-containing pair, giving at most T auxiliary-only support pairs. Each cleaning uses two auxiliary-only pairs, giving at most $2J_{\max}$ cleaning pairs. Their sum yields $P_{\max}$ in Eq. (5); actual pair activation satisfies $\sum_{p\in\mathcal{P}} v_p \le P_{\max}$.

Each PW pair denotes one uninterrupted occupancy from loading to unloading. A product-containing 2×2 pair participates in two adjacent production cycles but uses one pair index. A physical auxiliary wafer can be borrowed by another pair after return, so candidate-pair capacity differs from physical auxiliary inventory. The bound $Q_{\max} = T + J_{\max}$ limits total production-plus-cleaning cycles across the tool and also supplies the candidate capacity for each chamber.

Pair-to-cycle assignments. Let $\mathcal{Q}_c = \{1, \ldots, Q_{\max}\}$ be the candidate cycles of chamber c, with types 4×1, 2×2, and cleaning. The variable K_{cqm} selects type, and $U_{cq} = \sum_m K_{cqm} \in \{0, 1\}$ abbreviates cycle activation. The variable X_{pcq} selects participating pairs, and X^P_{pcq} identifies occupancy by a product-containing pair. Each active cycle contains two pairs with compatible type and chamber:

$$\begin{aligned}
&\sum_m K_{cqm} = U_{cq}, && \sum_p X_{pcq} = 2U_{cq},\\
&X_{pcq} \le z_{pc}, && X_{pcq} + \mu_{pm} \le 1 + K_{cqm},\\
&\sum_p X^P_{pcq} \ge K_{cq\zeta} + K_{cq\psi}, && U_{c,q+1} \le U_{cq},\\
&\sum_{c,q} U_{cq} \le Q_{\max}, && \sum_{c,q} K_{cq,\mathrm{cl}} \le J_{\max}.
\end{aligned} \tag{6}$$

The product-occupancy indicator is linked linearly by

$$X^P_{pcq} \le X_{pcq}, \quad X^P_{pcq} \le B_p, \quad X^P_{pcq} \ge X_{pcq} + B_p - 1. \tag{7}$$

The indicators B_p, B^ψ_p, X^P_{pcq} and the later $A^\psi_{pcq}, Z^\psi_{pcq}$ are logical auxiliaries determined by membership, type, and occupancy. Linear linking constraints are retained to avoid products of binary variables. Production cycles therefore contain a product

Table 2: Core decisions and necessary derived and linearization quantities

Symbol	Type	Definition and role
$a_{wpr}, v_p;\ d_{pr}$	Binary/derived	$a_{wpr} = 1$: product w occupies member position $r \in \{0,1\}$ of pair p; $v_p = 1$: pair p is active. Derived $d_{pr} = v_p - \sum_w a_{wpr}$ indicates auxiliary filling of that position.
μ_{pm}, z_{pc}	Binary	$\mu_{pm} = 1$: pair p has operation type m; $z_{pc} = 1$: it is assigned to chamber c, otherwise 0. Product recipe m_w remains an input.
B_p, B_p^{ψ}	Binary auxiliaries	$B_p = 1$: pair p contains a product; $B_p^{\psi} = 1$: it contains 2×2 products, otherwise 0. Superscript ψ is a recipe label.
$K_{cqm}, X_{pcq};\ U_{cq}$	Binary/derived	$K_{cqm} = 1$: cycle q in chamber c has type m; $X_{pcq} = 1$: pair p participates. Derived $U_{cq} = \sum_m K_{cqm} \in \{0,1\}$ indicates cycle activation.
$X_{pcq}^P, A_{pcq}^{\psi}, Z_{pcq}^{\psi}$	Binary auxiliaries	$X_{pcq}^P = 1$: product-containing pair p occupies cycle (c,q) (P labels products). $A_{pcq}^{\psi}, Z_{pcq}^{\psi} = 1$: its first/last exposure as a 2×2 product pair, respectively; otherwise 0.
b_o	Binary indicator	$b_o = 1$: candidate action or occupancy task o is active, otherwise 0; activation is linked to layout and optional transfers.
s_o, e_o	Event times	s_o, e_o: start/end of action or occupancy task o, in seconds. For fixed-duration actions, $e_o = s_o + d_o b_o$; holding endpoints are linked to physical events.
$\gamma_p^{\text{early}}, \gamma_p^{\text{late}}$	Binary	$\gamma_p^{\text{early}} = 1$: single-source auxiliary wafer in product–auxiliary pair p enters the lock first. $\gamma_p^{\text{late}} = 1$: it is picked up after product placement on AL. Superscripts label entry options; otherwise 0.
A_{pcq}, Z_{pcq}	Binary auxiliaries	$A_{pcq} = 1$: cycle (c,q) is the first occupied by pair p in chamber c; $Z_{pcq} = 1$: it is the last, otherwise 0.
$S_{cq}, E_{cq};\ n_{cq}$	Aliases/count	S_{cq}, E_{cq}: start/end event times of chamber cycle (c,q), in seconds. $n_{cq} \in \{0,\dots,H\}$: cumulative production count after that cycle, reset by cleaning.
$f_{pcq};\ \kappa_{cq}, \upsilon_{cq}$	Binary/derived	$f_{pcq} = 1$: pair p occupies the accessible front side in cycle (c,q). Derived $\kappa_{cq} = \sum_p (X_{pcq} - A_{pcq})$ marks one retained pair; $\upsilon_{cq} = U_{cq} - \sum_p (X_{pcq} - Z_{pcq})$ marks complete departure after the cycle.
$a_{vce}, q_{ce}^{\text{cl}}$	Segment indicators	$a_{vce} = 1$: production task v belongs to segment e of chamber c; $q_{ce}^{\text{cl}} = 1$: cleaning follows that segment. Both link to the cycle sequence; q^{cl} is an indicator name, not a cycle index.
$g_{j\tau}$	Binary	$g_{j\tau} = 1$: loan request $j = (p,r)$ uses physical auxiliary wafer τ, otherwise 0. Request and identity indices distinguish one use from one reusable wafer.
$h_{o\ell}, \gamma_{pa}$	Binary	$h_{o\ell} = 1$: task o selects physical unit ℓ; $\gamma_{pa} = 1$: pair p is transferred synchronously on eligible atomic leg a at a common location. Special $a =$ in denotes single-source accompanying entry.
$y_{oo'}^r$	Binary	For tasks o, o' sharing a unit of resource r, $y_{oo'}^r = 1$ selects o before o'; 0 selects the reverse. Superscript r identifies the resource; the prime distinguishes tasks.
$C_{\max}$	Nonnegative bound	$C_{\max}$: common upper bound on all product-return times $e_{w,\text{return}}$, in seconds, minimized in stage one. The labels return and max are semantic.
$\omega_\alpha, L_{\max}$	Expression/bound	ω_α: nonnegative event-time gap minus necessary duration for waiting item α; $L_{\max}$: common upper bound on all individual waits, in seconds.
v_{cq}	Nonnegative auxiliary	v_{cq}: upper bound on cadence variation $\lvert S_{c,q+1} - 2S_{cq} + S_{c,q-1} \rvert$, in seconds, for three consecutive production cycles centered at q in chamber c; defined only for $(c,q) \in \mathcal{I}_h$.
η_α	Nonnegative auxiliary	η_α: upper bound on waiting above soft limit $\bar{\omega}_\alpha$, in seconds; $\eta_\alpha \geq \max\{0, \omega_\alpha - \bar{\omega}_\alpha\}$.

pair, whereas cleaning cycles contain only auxiliary pairs. Prefix activation removes empty cycle indices without prescribing the order of operation types.

2.3. Action timing and precedence logic

Given physical routes and pairing relationships, activation, durations, precedence, and synchronization describe route feasibility and technical action order. Orders chosen to resolve shared-resource conflicts are addressed in Section 2.5.

Action activation and duration. For candidate action $o \in O$, let b_o indicate activation, s_o, e_o be start and end times, d_o be its prescribed duration, and M be a valid large constant. For a fixed-duration physical action, the end time follows from the start time and activation:

$$0 \leq s_o \leq M b_o, \qquad 0 \leq e_o \leq M b_o, \qquad e_o = s_o + d_o b_o. \tag{8}$$

Inactive actions have zero times; active actions last d_o. This form covers robot transfers, alignment, pumping, venting, rotation, recipe processing, auxiliary-wafer handling, and cleaning.

Route-specific precedence. Let $\mathcal{A}^{\text{int}}$ contain common chamber-internal precedence arcs, and let $\mathcal{A}_{is}(\gamma)$ contain the supply-and-return arcs for route task i, source region s, and transfer choices γ. The latter collectively denote paired transfers and single-source accompanying entry. The complete arc set is

$$\mathcal{A} = \mathcal{A}^{\text{int}} \cup \bigcup_{i \in \mathcal{I}} \bigcup_{\substack{s \in \mathcal{S}: \\ \chi_{is}=1}} \mathcal{A}_{is}(\gamma). \tag{9}$$

Equation (9) activates the route appropriate to each source and transfer choice and connects product routes, auxiliary supply/return routes, and chamber action chains.

For each required precedence arc $(o, o') \in \mathcal{A}$,

$$b_{o'} \leq b_o, \qquad s_{o'} \geq e_o + \delta_{oo'} - M(2 - b_o - b_{o'}). \tag{10}$$

A successor can execute only if its predecessor is active. When both are active, the successor starts after predecessor completion and the minimum interval $\delta_{oo'}$. These constraints connect wafer routes and loading–operation–unloading chains.

Stage links and paired-action synchronization. An individual wafer's stage must coincide with the stage selected for its batch, chain position, or cleaning task. For selection indicator x and reference interval $[s^{\text{ref}}, e^{\text{ref}}]$,

$$\begin{aligned} -M(1-x) \leq s_o - s^{\text{ref}} \leq M(1-x), \\ -M(1-x) \leq e_o - e^{\text{ref}} \leq M(1-x). \end{aligned} \tag{11}$$

When $x = 1$, the individual action and reference stage coincide; when $x = 0$, this candidate link is inactive. The link connects pair positions, product routes, auxiliary requests, and chamber operation windows in one time model.

Let w_1, w_2 be the two wafers in pair p. The variable γ_{pa} selects a paired transfer on an eligible atomic transfer leg a at a common location; s_{wa}, e_{wa} denote wafer w's start and end times on that leg. Paired transfer requires synchronization:

$$\begin{aligned} -M(1-\gamma_{pa}) \leq s_{w_1 a} - s_{w_2 a} \leq M(1-\gamma_{pa}), \\ -M(1-\gamma_{pa}) \leq e_{w_1 a} - e_{w_2 a} \leq M(1-\gamma_{pa}). \end{aligned} \tag{12}$$

When $\gamma_{pa} = 1$, the robot transfers the complete pair in one non-preemptive action. Otherwise, the two wafer actions still use the same robot and are ordered by Eq. (23). Actuator exclusivity counts coordinated actions: the two synchronized wafer events form one action and are not ordered against each other, while their common action remains exclusive with other actions. Each wafer retains its own hand-occupancy interval, so synchronization does not merge hand capacity.

Entry logic for a single-source product–auxiliary pair. Let w_1 be the product and w_2 the auxiliary wafer. The variable $\gamma_{p,\text{in}}$ chooses accompanying entry after both are collected. This route link does not require simultaneous pickup from different storage positions. The event labels pick, ALput, ALpick, LLput denote source pickup, placement on AL, retrieval from AL, and placement into the upper load lock. The indicators γ_p^{early} and γ_p^{late} choose direct auxiliary entry before product alignment or auxiliary pickup after product placement on AL. Exactly one of these and accompanying entry is selected:

$$
\begin{aligned}
\gamma_p^{\text{early}} + \gamma_p^{\text{late}} + \gamma_{p,\text{in}} &= 1, \\
\gamma_p^{\text{early}} = 1 &\Rightarrow e_{w_2,\text{LLput}} \le s_{w_1,\text{ALput}}, \\
\gamma_p^{\text{late}} = 1 &\Rightarrow s_{w_2,\text{pick}} \ge e_{w_1,\text{ALput}}, \\
\gamma_{p,\text{in}} = 1 \implies &\begin{cases}
e_{w_2,\text{pick}} \le s_{w_1,\text{ALput}}, \\
s_{w_2,\text{LLput}} = s_{w_1,\text{LLput}}, \\
e_{w_2,\text{LLput}} = e_{w_1,\text{LLput}}, \\
s_{w_1,\text{LLput}} \ge e_{w_1,\text{ALpick}}.
\end{cases}
\end{aligned}
\tag{13}
$$

For accompanying entry, ATR holds the auxiliary wafer before the product is placed on AL, and both enter the load lock together after alignment. The auxiliary wafer occupies one ATR hand from source pickup through completion of load-lock placement. The product occupies AL rather than an ATR hand during alignment and resumes hand occupancy after retrieval. Separate auxiliary supply directly links its pickup and load-lock placement. Neither option activates an AL-alignment event for the auxiliary wafer.

2.4. Production modes and cleaning

Section 2.2 determines which pairs occupy each cycle. This section specifies exposure counts, retention and departure between cycles, rotation and transfer order, and cleaning boundaries. All times use the physical-action representation in Section 2.3.

Production modes and continuous occupancy. A regular all-product 4×1 batch contains two product PW pairs. Four products receive one synchronous exposure and leave in last-in–first-out order; auxiliary wafers fill shortages. In 2×2, each product-containing pair occupies two adjacent production cycles in the same chamber, giving every product two exposures. For three product pairs, a leading auxiliary pair D is loaded, the chamber rotates, and P1 is loaded for production on D+P1. Rotation and chain exchanges then produce P1+P2, P2+P3, and P3+D. P1, P2, and P3 each receive two exposures; the trailing auxiliary pair preserves four-position occupancy. Figures 3 and 4 show the sequences.

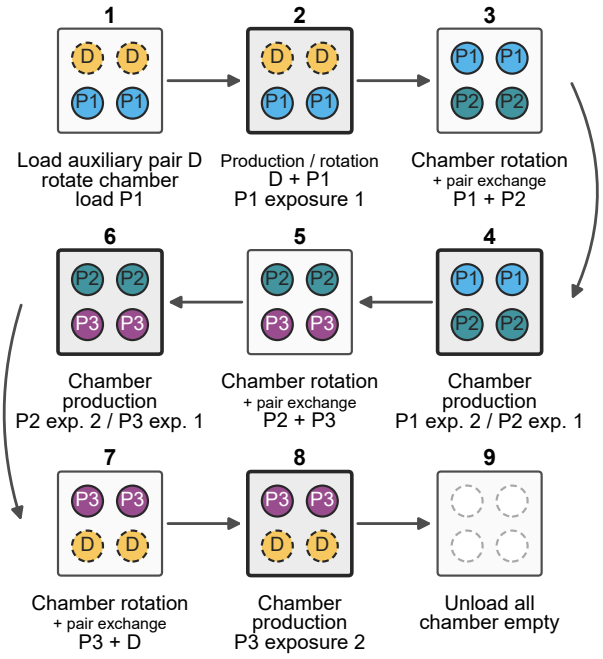


Figure 3: 2×2 chained production. Auxiliary pair D preserves full occupancy at the chain head and tail. Rotation and exchanges make each product pair participate in two adjacent production cycles, giving every product two exposures.

Binary A_{pcq}, Z_{pcq} identify the first and last cycles occupied by pair p in chamber c. With $X_{pc0} = X_{pc,Q_{\max}+1} = 0$,

$$
\begin{aligned}
&A_{pcq} \ge X_{pcq} - X_{pc,q-1}, && A_{pcq} \le X_{pcq}, \\
&A_{pcq} \le 1 - X_{pc,q-1}, && \\
&Z_{pcq} \ge X_{pcq} - X_{pc,q+1}, && Z_{pcq} \le X_{pcq}, \\
&Z_{pcq} \le 1 - X_{pc,q+1}, && \\
&\sum_{c,q} X_{pcq} = v_p + B_p^{\psi}, && \sum_{c,q} A_{pcq} = \sum_{c,q} Z_{pcq} = v_p, \\
&\sum_{p} A_{pcq}^{\psi} \le 1, && \sum_{p} Z_{pcq}^{\psi} \le 1.
\end{aligned}
\tag{14}
$$

Here $A_{pcq}^{\psi} = A_{pcq} \wedge B_p^{\psi}$ and $Z_{pcq}^{\psi} = Z_{pcq} \wedge B_p^{\psi}$ use the three-inequality linearization in Eq. (7). Each active pair enters and exits once, ensuring uninterrupted occupancy. Total occupancy $v_p + B_p^{\psi}$ gives one production cycle to a 4×1 product pair and two adjacent production cycles in one chamber to a 2×2 product pair. Each 2×2 cycle contains at most one product pair receiving its first exposure and one receiving its last, forming an alternating exchange chain. An auxiliary-only support pair occupies one production cycle, and a cleaning pair occupies one cleaning cycle.

Cycle durations and timing links. Let S_{cq}, E_{cq} be cycle start and end times, d_m the duration of operation type m, and $\overline{T}$ a

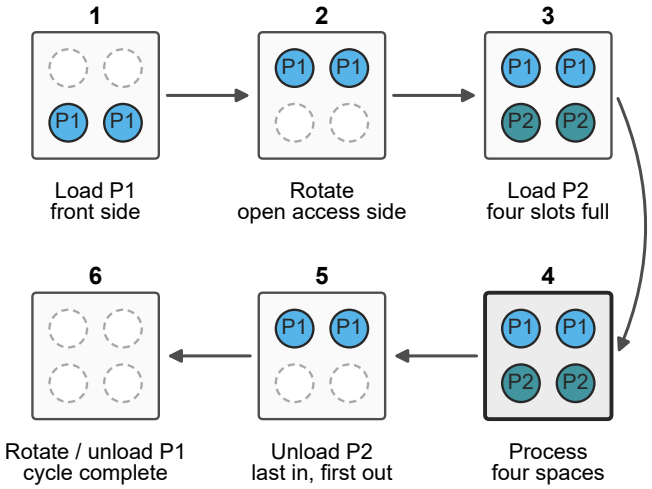


Figure 4: 4×1 parallel production. Two PW pairs are loaded successively around a rotation, filling all four positions for one synchronous exposure. Pairs leave in last-in–first-out order; auxiliary wafers fill product shortages.

valid time horizon:

$$\begin{aligned} E_{cq} - S_{cq} &= \sum_m d_m K_{cqm}, \\ 0 \le S_{cq}, E_{cq} &\le \overline{T} U_{cq}, \\ S_{cq} &\ge E_{c,q-1} - M(1 - U_{cq}) \quad (q > 1). \end{aligned} \tag{15}$$

Cycle durations are fixed; waiting occurs between actions. Default durations are 180 s for 4×1 production, 90 s for 2×2 production, and 500 s for cleaning. Transfer and rotation durations are counted through their separate events.

Front/back occupancy and rotating-chamber transfers. Binary f_{pcq} selects the pair at the robot-accessible front side during a cycle. The derived binary indicators $\kappa_{cq}, \upsilon_{cq}$ denote retention of one pair from the preceding cycle and complete departure after the current cycle. They follow from occupancy changes rather than independent choices:

$$\begin{aligned} \kappa_{cq} &= \sum_p (X_{pcq} - A_{pcq}), \\ \upsilon_{cq} &= U_{cq} - \sum_p (X_{pcq} - Z_{pcq}), \\ f_{pcq} &\le A_{pcq}, \qquad \sum_p f_{pcq} = U_{cq}, \\ f_{pc,q-1} &\ge X_{pcq} - A_{pcq} \quad (q > 1), \\ f_{pcq} &\ge B_p + K_{cq\psi} + X_{pcq} - 2 - \kappa_{cq}. \end{aligned} \tag{16}$$

A newly loaded pair occupies the front side. A retained pair rotates to the back and must have occupied the front in the preceding cycle. At a new 2×2 chain head, the product pair is loaded last into the front. Pair loading and unloading times are $s_p^{\text{load}}, e_p^{\text{load}}, s_p^{\text{unload}}, e_p^{\text{unload}}$. Rotation times before and after cycle (c, q) are $s_{cq}^{\text{rot}\pm}, e_{cq}^{\text{rot}\pm}$. These are aliases of physical events. Rotations rot− and rot+ are activated by U_{cq} and υ_{cq}, respectively, and last r_c. Loading links are

$$\begin{aligned} X_{pcq} = 1 &\Rightarrow S_{cq} \ge e_p^{\text{load}}, \quad s_p^{\text{unload}} \ge E_{cq}, \\ A_{pcq} = 1,\ f_{pcq} = 0 &\Rightarrow e_p^{\text{load}} \le s_{cq}^{\text{rot}-}, \\ A_{pcq} = 1,\ f_{pcq} = 1 &\Rightarrow s_p^{\text{load}} \ge e_{cq}^{\text{rot}-}, \quad S_{cq} = e_p^{\text{load}}, \\ \kappa_{cq} = 1 &\Rightarrow s_{cq}^{\text{rot}-} \ge E_{c,q-1}. \end{aligned} \tag{17}$$

When both pairs leave, post-cycle rotation starts after the chamber operation ends. Unloading after the last exposure obeys

$$\begin{aligned} Z_{pcq} = 1,\ f_{pcq} = 1 &\Rightarrow e_p^{\text{unload}} \le s_{cq}^{\text{rot}+}, \\ Z_{pcq} = 1,\ f_{pcq} = 0,\ \upsilon_{cq} = 1 &\Rightarrow s_p^{\text{unload}} \ge e_{cq}^{\text{rot}+}, \\ Z_{pcq} = 1,\ f_{pcq} = 0,\ \upsilon_{cq} = 0 &\Rightarrow s_p^{\text{unload}} \ge e_{c,q+1}^{\text{rot}-}. \end{aligned} \tag{18}$$

If the next cycle retains a pair, its new pair waits until the replaced back pair has been unloaded; otherwise, the next loading waits for both previous pairs to leave. Indicator constraints or valid big-M inequalities linearize these conditions, together with VTR exclusivity. Waiting remains explicit rather than being absorbed into rotation or transfer durations.

Cleaning triggers and action chains. Integer n_{cq} counts production cycles since the latest cleaning at completion of cycle q. With terminal candidate activation $U_{c,Q_{\max}+1} = 0$,

$$\begin{aligned} 0 \le n_{cq} \le H U_{cq}, &\qquad n_{c0} = 0, \\ K_{cq,\text{cl}} = 1 &\Rightarrow n_{cq} = 0, \\ U_{cq} = 1,\ K_{cq,\text{cl}} = 0 &\Rightarrow n_{cq} = n_{c,q-1} + 1, \quad n_{c,q-1} \le H - 1, \\ K_{cq,\text{cl}} = 1 &\Rightarrow 1 \le n_{c,q-1} \le H, \\ n_{cq} &\le H - 1 + H U_{c,q+1}, \\ K_{c1,\text{cl}} = 0, &\qquad K_{cq,\text{cl}} + K_{c,q-1,\text{cl}} \le 1 \quad (q > 1). \end{aligned} \tag{19}$$

Each production cycle increments the count by one, and cleaning resets it to zero. Cleaning is mandatory at H. After 1 to $H-1$ production cycles, it may be selected at a valid boundary satisfying full occupancy, uninterrupted exposures, and resource constraints, with no restriction on the next production mode. Thus successive cleanings are separated by 1 to H production cycles. A final segment below H need not be followed by cleaning; reaching H requires terminal cleaning. Equations (6) and (14) keep the two exposures of each 2×2 product pair adjacent, so cleaning cannot split them.

Cleaning segments e follow from cleaning boundaries in the cycle-type sequence. Let $\mathcal{V}_c$ contain production tasks, each being a 4×1 batch or a 2×2 production cycle. The indicator a_{vce} assigns task v to segment e of chamber c, and q_{ce}^{cl} indicates cleaning after that segment. These indicators are linked to the cycle structure rather than providing independent cleaning-count or segmentation decisions. Internal segments end with cleaning; terminal cleaning follows Eq. (19).

Task v uses event times s_v, e_v. The complete cleaning task following segment e has endpoints $s_{ce}^{\text{cl}}, e_{ce}^{\text{cl}}$; its stage times are $s_{ce}^{\text{cl},\nu}, e_{ce}^{\text{cl},\nu}$ for $\nu \in \{FL, BL, P, BU, FU\}$, denoting front loading, back loading, cleaning treatment, back unloading, and front unloading. These aliases refer to physical events; the complete

task spans front-loading start through front-unloading completion. The timing relationships are

$$\begin{aligned} s_v &\geq e^{\mathrm{cl}}_{c,e-1} - M(1 - a_{vce}) && (e > 1), \\ e_v &\leq s^{\mathrm{cl}}_{ce} + M(1 - a_{vce}) + M(1 - q^{\mathrm{cl}}_{ce}), \\ s^{\mathrm{cl,BL}}_{ce} &\geq e^{\mathrm{cl,FL}}_{ce} + r_c - M(1 - q^{\mathrm{cl}}_{ce}), \\ s^{\mathrm{cl,P}}_{ce} &\geq e^{\mathrm{cl,BL}}_{ce} - M(1 - q^{\mathrm{cl}}_{ce}), \\ s^{\mathrm{cl,BU}}_{ce} &\geq e^{\mathrm{cl,P}}_{ce} - M(1 - q^{\mathrm{cl}}_{ce}), \\ s^{\mathrm{cl,FU}}_{ce} &\geq e^{\mathrm{cl,BU}}_{ce} + r_c - M(1 - q^{\mathrm{cl}}_{ce}). \end{aligned} \tag{20}$$

Segment e starts after the preceding cleaning task ends and ends before the next cleaning begins. A complete cleaning task includes loading auxiliary wafers on both sides, rotation, cleaning treatment, and unloading in reverse order. Its durations and resource occupancy constrain VTR, the chamber, and all four auxiliary wafers.

Once a solution is given, 4×1 batch sides, 2×2 product-pair chain positions, and cleaning segments can be recovered from X, A, Z, f and cycle types without additional independent position variables. Fixed-layout experiments additionally fix these positions and service orders and permit filling only in the final short batch: if later same-type batches exist, the current 4×1 batch contains two product pairs. The joint model allows auxiliary filling in any valid production cycle.

2.5. Auxiliary-wafer reuse and robot/module resources

Wafer routes and chamber rules must also respect physical-resource availability. Each auxiliary loan is first assigned a physical identity, followed by physical-hand and module-slot assignments. Resource orders and residency limits coordinate their occupancy.

Auxiliary identities and reuse. Each auxiliary member position defines a loan request $j = (p, r)$, activated directly by d_{pr} for product–auxiliary, auxiliary-only filling, or cleaning pairs. Membership and cycle choices link its supply and return events. Physical auxiliary occupancy extends from storage pickup through completed return.

All chambers access one physical auxiliary inventory. The set $\mathcal{T}_c$ contains identities accessible to chamber c; loans of the same identity must be exclusive across chambers as well. Each active request selects one physical wafer:

$$\sum_{\tau \in \mathcal{T}_c} g_{j\tau} = d_{pr}, \qquad j = (p, r) \in \mathcal{J}^D_c. \tag{21}$$

The assignment $g_{j\tau}$ also marks occupancy of the borrowed physical identity. Shared-resource constraints order successive loans, permitting reuse only after unloading and completed return to storage. This condition governs reuse of the same physical wafer; it does not imply that every return delays subsequent processing. Another available wafer can be selected instead. A subsequent loan assigned the same wafer must wait for its return, but whether this wait delays product processing or batch completion also depends on other precedence relations and handling-resource timing. Identity choices determine which cross-chamber loans require ordering: even with identical inventory, product pairing, chamber assignment, and cycle structure, different identities may change concurrent production intervals. Section 4.9 quantifies this effect with product structure controlled.

Robot hands and module capacities. ATR has two hands, VTR has four, and each rotating chamber has four wafer positions. At any instant, each ATR or VTR executes at most one coordinated transfer, and each chamber executes at most one unified loading, unloading, rotation, or chamber-operation stage. Their actuators are therefore exclusive action resources. Individual hand assignments express robot carrying capacity, and pair occupancy expresses chamber capacity; actuator exclusivity does not represent wafer count. AL is also exclusive. The upper and lower load locks contain slots U1/U2 and L1/L2, respectively, and every physical auxiliary wafer is a reusable identity. Each occupancy interval selects one allowed unit, whether a physical hand, an auxiliary identity, or an exclusive actuator. For occupancy task o,

$$\sum_{\ell \in \Gamma_o} h_{o\ell} = b_o. \tag{22}$$

Binary b_o activates the task, $h_{o\ell}$ selects unit ℓ, and Γ_o is the set of allowed units. Inactive tasks select no unit. Synchronized wafers retain separate hand occupancies, with synchronization enforced by Eq. (12). Full chamber occupancy follows from pair assignments. Equation (23) prevents overlapping occupancy of any physical unit. If Γ_o is a singleton, $h_{o\ell} = b_o$ can be substituted directly. Auxiliary-identity occupancy uses $g_{j\tau}$ without a duplicate assignment decision.

Upper-slot occupancy extends from ATR placement start through VTR pickup completion plus reset time. Lower-slot occupancy extends from VTR placement start through ATR pickup completion plus reset time. Reset limits reuse of that slot and may overlap subsequent robot travel.

VTR has dirty hands D_1, D_2 and clean hands C_1, C_2. Product holding from upper-lock pickup to chamber placement permits $\Gamma_o = \{D_1, D_2\}$; holding after all required exposures, from chamber pickup to lower-lock placement, permits $\Gamma_o = \{C_1, C_2\}$. Auxiliary input and return permit all four hands. Each holding interval lasts from pickup start through placement completion and retains the same physical hand selected by Eq. (22). Same-hand intervals are exclusive with zero hand-switching gap; robot travel is separately included through actuator constraints. Each hand class therefore holds at most two wafers, with auxiliary wafers competing for both classes. A 2×2 product remains inside the chamber between its two exposures and leaves on a clean hand after the second. Paired exchange permits dirty hands to hold an incoming pair while clean hands remove a completed pair, provided every auxiliary wafer is also assigned a free physical hand.

Resource exclusivity and location-dependent travel. Resource orders are either prescribed or selected. Precedence arcs and predefined service chains impose the former; order variables resolve other conflicts. Fixed-layout pure-4×1 experiments prescribe aligner and load-lock service orders and, when

the number of candidate batches does not exceed the cleaning threshold, a VTR batch-transfer chain. These restrictions apply only to fixed-layout experiments; the joint model chooses the remaining resource orders.

For tasks o, o' competing for unit ℓ of resource r,

$$\begin{aligned} s_{o'} &\geq e_o + \rho^r_{oo'} - M(1 - y^r_{oo'}) - M(1 - h_{o\ell}) - M(1 - h_{o'\ell}), \\ s_o &\geq e_{o'} + \rho^r_{o'o} - M y^r_{oo'} - M(1 - h_{o\ell}) - M(1 - h_{o'\ell}). \end{aligned} \tag{23}$$

When both tasks select the unit, binary $y^r_{oo'}$ chooses their order. The parameter $\rho^r_{oo'}$ includes repositioning or reset time. The constraint covers ATR, AL, load locks, VTR, chambers, and physical auxiliary identities, preventing conflicting actuator actions as well as premature reuse of slots or wafers.

Travel times are specified by location pairs. For actions whose chamber is chosen by z_{pc}, the relevant travel gap is activated by the selected chamber combination rather than a predetermined location. For example, when active exclusive chamber actions o, o' serve pairs p, p',

$$z_{pc} = z_{p'c'} = 1,\ y^{\mathrm{VTR}}_{oo'} = 1 \ \Rightarrow\ s_{o'} \geq e_o + \rho^{\mathrm{VTR}}_{cc'}. \tag{24}$$

The parameter $\rho^{\mathrm{VTR}}_{cc'}$ is the travel time from chamber c to c'. Reverse order is handled symmetrically; fixed-location actions use Eq. (23) directly.

Residency limits and timing feasibility. For the events at which wafer w enters and leaves constrained equipment region r,

$$0 \leq s_{\mathrm{out}(w,r)} - e_{\mathrm{in}(w,r)} \leq \overline{R}_{wr}. \tag{25}$$

This bounds physical residency in a load lock, chamber, or constrained transport segment. Load-lock slot occupancy ends after the wafer is picked up from the opposite side; subsequent pressure reset may overlap robot travel, while the next incompatible load-lock action must await reset completion.

Residency limits enforce temporal feasibility in the corresponding segments; they do not require same-recipe wafers to have identical residency durations. Necessary processing and handling times can also differ by mode and segment. A study of residency uniformity would need a dispersion measure for comparable wafers with the same recipe and segment, evaluated or optimized directly. Reduced waiting is not a substitute for that test.

The precedence constraints in Eq. (10) and resource-order constraints in Eq. (23) jointly prevent modeled deadlocks. Construct a directed graph whose nodes are active actions and whose arcs are technical precedence or selected resource orders. A circular wait among robots, load locks, chambers, or auxiliary wafers creates a directed cycle. Summing its time inequalities accumulates a positive duration or switching increment for actual occupancy actions, requiring an action to start strictly later than itself. Such an order is infeasible. Together with explicit load-lock slots, full chamber occupancy, and return-before-reuse requirements, time feasibility excludes circular waiting among the modeled actions and permits wafers to complete their routes.

2.6. Makespan-first two-stage optimization

The preceding constraints define the joint feasible region $\mathcal{F}$, with ξ the vector of retained variables. Stage one optimizes product-return makespan. Stage two improves waiting and production cadence under the first-stage makespan bound and principal discrete structure.

Stage one: minimize product-return makespan. The return-action end time $e_{w,\mathrm{return}}$ directly denotes product w's return to its original LP:

$$C_{\max} \geq e_{w,\mathrm{return}}\ (w \in \mathcal{W}), \qquad \min_{\xi \in \mathcal{F}} C_{\max}. \tag{26}$$

Makespan is measured by the last product's completion of its entire route. Cleaning between production segments delays subsequent products through resource and precedence constraints and thus affects makespan. Terminal cleaning after the last production remains in the action sequence but its completion is not a direct lower bound on $C_{\max}$. It may still delay returns from another chamber through the shared VTR, so indirect effects require the complete schedule. WPH uses product-return $C_{\max}$ as its denominator. The joint feasible region includes pairing, chamber assignment, cycle occupancy, and complete physical timing; fixed-layout experiments add their layout and service-order restrictions.

Stage two: restrict the refinement domain. Let $\widehat{\xi}$ be the best first-stage incumbent and $\widehat{C}$ its product makespan. The symbol ξ_d denotes component d, and hatted quantities are constants in stage two. The index set $\mathcal{D}_{\mathrm{fix}}$ covers fixed pairing, chamber assignments, cycle occupancy, auxiliary identities, hand assignments, accompanying-entry choices, and resource orders. With nonnegative makespan tolerance ε_C, impose

$$\xi_d = \widehat{\xi}_d \quad (d \in \mathcal{D}_{\mathrm{fix}}), \qquad C_{\max} \leq \widehat{C} + \varepsilon_C. \tag{27}$$

These constraints retain principal discrete structure and the makespan bound, while same-location atomic-transfer synchronization and continuous times remain adjustable. Stage two can start from a first-stage incumbent even without a proof of optimality.

Let Q^w index the waiting items and ω_α be the nonnegative interval remaining after subtracting necessary action or travel time between the corresponding events. Total waiting enters the objective directly as $\sum_\alpha \omega_\alpha$. The experimental label L_{sum} is only a summary statistic for this sum, not an independent variable. The quantity $L_{\max}$ bounds the largest individual wait. Simultaneous waits of different wafers are counted separately. Each ω_α is an event-time difference expression rather than an independent waiting decision:

$$L_{\max} \geq \omega_\alpha \geq 0, \qquad \alpha \in Q^w. \tag{28}$$

Cadence uses cycle start times S_{cq} directly. The set $\mathcal{I}_h$ contains the central indices (c, q) of three consecutive production cycles within the same chamber and 2×2 chain. Both boundaries retain a pair, so $\kappa_{cq} = \kappa_{c,q+1} = 1$; differences do not cross cleaning or chain boundaries. Fixing cycle structure in stage two fixes this set. The difference between successive start-to-start intervals is $S_{c,q+1} - 2S_{cq} + S_{c,q-1}$. Nonnegative v_{cq} bounds

its absolute value linearly without a separate segment clock or difference variable:

$$\begin{aligned} v_{cq} &\geq S_{c,q+1} - 2S_{cq} + S_{c,q-1}, \\ v_{cq} &\geq -S_{c,q+1} + 2S_{cq} - S_{c,q-1} \quad ((c,q) \in \mathcal{I}_h). \end{aligned} \tag{29}$$

When soft waiting limits $\bar{\omega}_\alpha$ are enabled, nonnegative η_α bounds excess waiting:

$$\eta_\alpha \geq \omega_\alpha - \bar{\omega}_\alpha, \qquad \eta_\alpha \geq 0. \tag{30}$$

With nonnegative weights $\lambda_1, \ldots, \lambda_4$, the second-stage objective is

$$\begin{aligned} \min \ \lambda_1 \sum_{\alpha \in Q^w} \omega_\alpha + \lambda_2 L_{\max} \\ + \lambda_3 \sum_{(c,q) \in \mathcal{I}_h} v_{cq} + \lambda_4 \sum_{\alpha \in Q^w} \eta_\alpha. \end{aligned} \tag{31}$$

The terms penalize total waiting, maximum individual waiting, variation in adjacent production-cycle start intervals, and soft-limit excess. The experiments use $(\lambda_1, \lambda_2, \lambda_3) = (1, 10, 25)$ and $\varepsilon_C = 10^{-6}$. Disabling soft waiting limits omits their variables and penalty.

At an optimal second-stage solution with strictly positive corresponding weights, the auxiliary bounds are tight: $L_{\max} = \max_\alpha \omega_\alpha$, $v_{cq} = |S_{c,q+1} - 2S_{cq} + S_{c,q-1}|$, and $\eta_\alpha = \max(0, \omega_\alpha - \bar{\omega}_\alpha)$. Time-limited feasible solutions may retain slack, so reported metrics are recomputed from actual event times. Cadence variation is zero for three equally spaced production starts. Soft waiting limits allow penalized excess; residency limits remain hard constraints.

Waiting, cadence, and JIT.. Equation (28) reduces event intervals after subtracting necessary durations, whereas Eq. (29) reduces changes in consecutive cycle-start intervals within one chain. Neither directly minimizes differences in wafer residency. Even equally spaced cycle starts can yield different residency durations through unequal unloading waits. Reducing waiting and irregular coordination supports JIT-oriented operation, but the model does not include demand-matching targets, earliness/tardiness costs, or a residency-uniformity objective.

3. Solution framework for time-limited feasible scheduling

The algorithmic task is to return a schedule satisfying the complete physical constraints within specified time and memory budgets, then improve its objective and optimality bounds. Updated equipment parameters can define a new deterministic instance for this workflow; an old schedule is not assumed to remain feasible under new parameters. Limited computation reflects both the complexity of joint scheduling and the need for timely decisions after updates, although no real equipment rescheduling deadline is measured here. SPBS targets feasible initialization, timing refinement improves the selected structure, and learned cut selection is an experimentally assessed search component rather than a prerequisite for mode-selection conclusions.

Section 2 links membership and cycle occupancy to complete wafer routes in a joint model. The solution framework combines structured initialization, MILP search, and timing refinement for both the fixed-configuration special case and the joint model. The fixed-layout benchmark supports training and component comparisons; the joint model evaluates search with the frozen learned policy. Primal feasibility requires coordination of full occupancy, dual-source logistics, and shared-resource order, whereas dual-bound improvement depends on valid inequalities acting on discrete choices, continuous times, and their coupling. We exploit these structures through a structure-preserving binary skeleton (SPBS) for initial feasible scheduling and RDMCT-HBS for cut selection. SPBS supplies a feasible starting point, and RDMCT-HBS determines cut counts and insertion order. SCIP generates valid cuts, checks feasibility, maintains bounds, and certifies optimality [32, 33].

The solution process comprises initial-schedule construction and validation, candidate-cut generation, count prediction, ordered-subset search, and role-submodular completion, followed by continued branch-and-bound search. All selected cuts are valid inequalities for the original model and retain their coefficients. Cut selection therefore controls the strengthening of the linear relaxation while preserving the original integer feasible set.

3.1. SPBS initial solution

For the evaluated fixed-configuration special case, SPBS retains its pairing, chamber and cleaning layout, constructs a skeleton for selectable resource orders and slots, and then solves for event times in a restricted MILP. Product wafers are paired in input order. The 4×1 pairs are mapped sequentially to chambers, batches, and front/back positions, whereas the 2×2 pairs are assigned round-robin to chamber-chain positions. This determines batch-use, tail-batch, used-position, and last-position indicators. Cleaning counts partition each chamber sequence into process segments: 4×1 batches are assigned to their cleaning epochs, while a 2×2 chain ends its current segment with auxiliary wafers when the cleaning threshold is reached and resumes after cleaning with a new auxiliary-bounded head. Processing assignments, chamber membership, and cleaning boundaries follow from the benchmark configuration; SPBS coordinates auxiliary occupancy and event times within that layout.

At the resource level, SPBS derives a conservative order from the same physical route. ATR first schedules 4×1 inputs and returns across chambers and stages the first 2×2 pair of a chamber immediately after that chamber's final full batch. It then advances chain inputs and returns segment by segment. Deterministic odd/even load-lock slots break slot symmetry, and the VTR order is aligned with ATR batch progression and cleaning actions. Let $\mathcal{F}_q$ be the binary variables fixed in relaxation level q, with skeleton value $\widehat{x}_j$. The restricted feasibility subproblem imposes

$$\ell_j^{(q)} = u_j^{(q)} = \widehat{x}_j, \qquad j \in \mathcal{F}_q \tag{32}$$

Here j indexes scalar MILP variable x_j, and $q \in \{1, \ldots, 5\}$ indexes the initialization relaxation level. The bounds $\ell_j^{(q)}, u_j^{(q)}$

apply to x_j at that level, and $\widehat{x}_j$ is its skeleton value. The fixed-index set $\mathcal{F}_q$ differs from feasible region $\mathcal{F}$ in Section 2; q here is not a chamber-cycle index.

Equation (32) fixes only the skeleton choices at the current level. Start, completion, and waiting times, together with resource orders not fixed at that level, remain free.

SPBS progressively relaxes fixed-variable conditions through five levels, with nested fixed sets $\mathcal{F}_1 \supseteq \mathcal{F}_2 \supseteq \cdots \supseteq \mathcal{F}_5$. For unchanged skeleton values, releasing these conditions expands the restricted feasible set. If a level yields no feasible solution within its allocated budget, the search proceeds to the next level. The first level fixes product and batch assignments, activation states, cleaning segments, load-lock slots, and ATR, AL, LL, and VTR orders. The second releases VTR order; the third also releases load-lock order; the fourth releases all remaining resource orders while retaining structural assignments and slots; and the fifth retains only assignments, activation states, and cleaning segments. For instances with at least 32 wafers, the search starts at the fourth or fifth level. Under a tight waiting-time limit, the original feasible set without additional skeleton restrictions is also considered.

Each level solves a restricted feasibility subproblem with a zero objective and terminates at the first feasible solution. These subproblems use feasibility emphasis, aggressive heuristics, and fast presolve, without cut separation. The resulting principal discrete decisions and continuous event times are then retained in the full model while auxiliary variables associated with schedule stability are completed. A schedule satisfying all original-model constraints becomes the initial incumbent of the main problem. The main problem restores the bounds temporarily tightened by SPBS while retaining the original layout equalities; its feasible set is the complete set conditional on the prescribed layout. In the experiments, SPBS schedules are constructed offline and checked against their corresponding models; the PDI integration interval and main-solve timing both exclude offline initial-schedule construction.

Hierarchical initialization for the joint model starts from a schedule satisfying complete wafer routes and physical-hand eligibility and improves the skeleton with product makespan as its objective. When enabled, its five levels retain the principal discrete choices, release VTR action orders, release load-lock orders, release the remaining resource orders, and finally retain only structural choices such as pairing, cycle activation, and cycle type. Paired-action synchronization and continuous times remain adjustable at every level. Each level receives at most one fifth of the initialization budget and passes its best complete feasible assignment to the next level. All temporary restrictions are removed before joint main search, which again optimizes pairing, chamber assignment, and resource order. This makespan-improvement initialization differs from the zero-objective feasibility search used for fixed-layout SPBS; construction and main-solve budgets are accounted for separately.

3.2. HEM baseline and role-aware representation

Prior learning-based cut selectors address generic candidate sets with reinforcement learning, learned ranking, and look-ahead imitation [28–30]. HEM further identifies three coupled decisions—which cuts, how many cuts, and in what order—and handles them with a high-level count policy and a low-level sequence policy [31]. We retain HEM's hierarchical decomposition and pointer decoding and use its 13D sequence model as the direct baseline. Training instances, budgets, random seeds, candidate limits, and hidden dimensions are shared while representation, decoding, and post-selection completion are compared. HEM was subsequently extended to hierarchical sequence/set models [34]; HEM-13D here is a controlled reproduction of the 2023 sequence model.

The 13D HEM candidate vector comprises objective parallelism, efficacy, normalized support, integral-support ratio, normalized violation, the mean/maximum/minimum/standard deviation of row coefficients, and the corresponding four objective-coefficient statistics. These features describe immediate LP quality and numerical scale. They do not distinguish two similarly efficacious rows when one acts on discrete route choices and the other on continuous event times. In the present model, that distinction corresponds directly to changing a process/resource combination versus tightening the timing of an existing combination.

Starting from the 13 generic HEM features, variables are partitioned by their mathematical attributes into binary, general integer, objective-active continuous, continuous auxiliary, and other roles. The coefficient mass of role g in cut i is

$$p_{ig} = \frac{\sum_{j:x_j \in g} |a_{ij}|}{\sum_j |a_{ij}| + \varepsilon}. \tag{33}$$

In Eq. (33), i indexes candidate cuts and j indexes original scalar MILP variables x_j; a_{ij} is the coefficient of x_j in cut i. The index $g \in \mathcal{G}$ denotes a mathematical variable role, p_{ig} its absolute coefficient-mass fraction in cut i, and $\varepsilon > 0$ a denominator-stabilization constant. The five roles are binary, general integer, objective-active continuous, continuous auxiliary, and other; $B, I, C_{\text{obj}}, C_{\text{aux}}$ label the first four. The cut coefficient a_{ij} differs from wafer-membership indicator a_{wpr} in Section 2.

Equation (33) is the fraction of absolute cut-coefficient mass carried by a mathematical variable role. Beyond the five role masses, the added features are mass on variables with two finite global bounds, positive-coefficient mass, discrete–continuous coupling $4(p_{iB} + p_{iI})(p_{iC_{\text{obj}}} + p_{iC_{\text{aux}}})$, the largest non-residual role mass, and normalized role entropy. The resulting vector has 23 dimensions. Role assignment depends only on variable types, objective coefficients, and global bounds. The representation is therefore invariant to variable renaming that preserves these attributes and the cut coefficients.

HEM uses a hierarchical "count policy + ordered-subset policy" decomposition: its high level predicts the number of cuts from a sequence representation, while its low level uses an LSTM encoder–decoder with pointer attention to select cuts sequentially [31, 35]. Building on this hierarchy, the proposed selector introduces a role-aware grouped gated encoder before the LSTM, a role-submodular completion step after the pointer sequence, and width-three beam search during formal inference, which together form the complete RDMCT-HBS selection pipeline.

3.3. Role-aware encoding and hierarchical sequence decoding

Let m_t be the number of candidates retained at separation round t. For candidate cut i, write the 23D input as $x_i = [x_i^H; x_i^R]$, where $x_i^H \in \mathbb{R}^{13}$ contains the generic HEM features and $x_i^R \in \mathbb{R}^{10}$ contains the proposed role-aware features. The full vector is first layer-normalized; the two feature groups are then projected separately into the common hidden space and fused by an elementwise gate generated from all 23 features:

$$\begin{aligned} \widetilde{x}_i &= \mathrm{LN}(x_i), \\ u_i^H &= W_H \widetilde{x}_i^H + b_H, \qquad u_i^R = W_R \widetilde{x}_i^R + b_R, \\ g_i &= \sigma(W_g \widetilde{x}_i + b_g), \qquad z_i = g_i \odot u_i^H + (1 - g_i) \odot u_i^R. \end{aligned} \tag{34}$$

The index i identifies a candidate cut. Superscripts H, R label the generic HEM and role-feature branches. The operator LN produces the layer-normalized vector $\widetilde{x}_i$, with components $\widetilde{x}_i^H, \widetilde{x}_i^R$. The projected vectors u_i^H, u_i^R, gate g_i, and fused vector z_i have 128 components. Matrices W_H, W_R, W_g and biases b_H, b_R, b_g parameterize the two projections and gate; their subscripts identify network components. The function $\sigma(\cdot)$ is sigmoid and $\odot$ is elementwise multiplication. Weights and biases are learned during training and fixed during inference. This σ is unrelated to the source-configuration input in Section 2.

Here $z_i \in \mathbb{R}^{128}$ is the fused candidate embedding. Unlike a single projection of the 23D vector, the two branches preserve generic LP quality and mathematical-role structure separately, while g_i determines their candidate-specific contribution in each hidden dimension. Thus two cuts with similar efficacy but different concentration on discrete route variables or continuous timing variables can remain distinguishable. This grouped gated adapter is a structural change to HEM's input representation introduced in this work.

The fused candidates, in the deterministic order established by Step 1, are passed through a single-layer LSTM encoder:

$$(h_i, c_i) = \mathrm{LSTM}(z_i, h_{i-1}, c_{i-1}), \qquad i = 1, \ldots, m_t. \tag{35}$$

The index t denotes a separation round, m_t its candidate count after truncation, and $i = 1, \ldots, m_t$ the encoding order. Vectors h_i, c_i are the hidden and memory states after candidate i, and h_{i-1}, c_{i-1} are the preceding states; both have 128 components. The memory state c_i is not chamber index c.

The states h_i turn local cut descriptors into context-dependent candidate representations and are used by both the high-level retained-count policy and the low-level ordered selector. A 128D hidden state is used to encode the ordered candidates into a contextual sequence representation. Fixing the input order also makes the encoded sequence reproducible for an identical candidate pool.

At low-level decoding step r, the decoder state d_r formed from the selected prefix and all encoder states h_i are used for one glimpse attention followed by pointer attention. Let $M_{ri} = -\infty$ if candidate i already appears in the prefix and $M_{ri} = 0$ otherwise. Then

$$\begin{aligned} a_{ri} &= v_g^\top \tanh(W_e h_i + W_d d_r) + M_{ri}, \\ \alpha_{ri} &= \mathrm{softmax}_i(a_{ri}), \\ \bar{h}_r &= \sum_i \alpha_{ri} h_i, \\ b_{ri} &= v_p^\top \tanh(U_e h_i + U_g \bar{h}_r) + M_{ri}, \\ P(i_r = i \mid i_{1:r-1}, S_t) &= \mathrm{softmax}_i(b_{ri}). \end{aligned} \tag{36}$$

Subscripts r, i denote decoding step and candidate cut. The quantities $a_{ri}, \alpha_{ri}, \bar{h}_r, b_{ri}$ are the glimpse score, its candidate-normalized attention weight, the weighted candidate summary, and the pointer score. Matrices W_e, W_d project encoder and decoder states; U_e, U_g project encoder states and the glimpse summary. Vectors v_g, v_p are trainable scoring vectors; subscripts e, d, g, and p label network components. The candidate mask M_{ri} differs from time constant M. The selected cut at step r is i_r, its preceding ordered prefix is $i_{1:r-1}$, and S_t is the encoded candidate state for round t. The notation $P(\cdot \mid \cdot)$ denotes a conditional selection probability.

Equation (36) makes the sequential dependence explicit. The selected prefix updates the decoder state and the next conditional distribution. Masking excludes previously selected cuts, and the glimpse summarizes the candidate pool for the current prefix. The resulting sequence model combines contextual candidate encoding, prefix-aware pointer decoding, and the proposed representation and completion stages into one role-aware selector.

Training keeps stochastic sampling from the policy for exploration. During formal inference, instead of committing to one path at every step, the method maintains $B = 3$ partial sequences. For parent beam $b = (i_1, \ldots, i_{r-1})$ and an unselected extension j, the accumulated score is

$$L_r(b \oplus j) = L_{r-1}(b) + \log P(i_r = j \mid b, S_t), \tag{37}$$

Here b denotes a parent prefix and j an unselected candidate cut; $b \oplus j$ appends j. The score L_r is joint log probability for a length-r prefix, with $L_0 = 0$ for the empty prefix. The symbols B and $\mathcal{B}_r$ denote beam width and retained prefixes at step r, not production batches.

All parent-beam–candidate extensions are compared and the three largest L_r values are retained. After the high-level target length k_t is reached, the best sequence is passed to the completion stage. This beam procedure preserves multiple high-probability partial sequences during decoding, reduces early commitment, and works jointly with the subsequent policy-anchored role-submodular completion to inject variable-role structure into the final selected set.

3.4. Stepwise selection from candidate pool to ordered cuts

At separation round t, the method executes the following sequence.

Step 1: form the candidate pool. SCIP first generates and validates candidate rows C_t with its own separators. To control the candidate-state dimension and evaluation cost, at most 128

candidates are retained in descending order of cut efficacy. Ties are broken by original index for deterministic input. Forced cuts remain under solver control and are not placed in the learned ranking.

Step 2: compute and encode the state. The 13 HEM and 10 role-aware features are extracted for every row, together with role masses, dominant role, and structural entropy. Equation (34) then performs grouped gated fusion and Eq. (35) produces contextual candidate states. The encoded candidate set is S_t.

Step 3: determine the retained count. The high-level policy outputs a tanh-bounded action in $[-1, 1]$, which is mapped to a retained fraction ρ_t. The fraction is then clipped by the global 0.2 cap, SCIP's round-specific limit, and the limit of 16 cuts, producing k_t. When enough candidates exist, at least two rows are retained to maintain a nonempty selected subset in each round.

Step 4: construct the candidate order. At low-level step r, the pointer policy conditions on S_t and the selected prefix $i_{1:r-1}$ to score all remaining candidates. Selected indices are masked with negative infinity and cannot be repeated. The joint high- and low-level policy is

$$\pi_\theta(k_t, i_{1:k_t} \mid S_t) = \pi_\theta^H(k_t \mid S_t) \prod_{r=1}^{k_t} \pi_\theta^L(i_r \mid S_t, i_{1:r-1}). \tag{38}$$

The parameter vector is θ, the retained count at round t is k_t, and $i_{1:k_t}$ is the ordered cut sequence. The policies $\pi_\theta^H, \pi_\theta^L$ are the high-level count and low-level conditional selection policies; superscripts H and L denote their levels. The retained fraction ρ_t is mapped from the high-level output; subscript t denotes the separation round.

Equation (38) means that the high level determines the sequence length and the low level then chooses distinct candidates within that length. Each earlier cut changes both the remaining action set and the decoder state used by later probabilities.

Step 5: beam decoding. The low-level policy samples stochastically during training. At test time, beam search of width three is used. At step r, every partial sequence is extended by every unselected candidate; joint log-probabilities are accumulated, and the three highest-scoring items over all parent-beam–next-candidate pairs are retained. After k_t steps, the best sequence is returned. Unlike one-step greedy decoding, the beam retains an initially lower-ranked prefix whose later joint probability can be higher. This fixed-width neural construction follows the broader use of beam decoding in neural combinatorial optimization [36].

Step 6: role-submodular completion. A sequence based only on joint probability can still repeatedly cover one variable role. The first $\lceil 0.25k_t \rceil$ pointer candidates are retained as policy anchors. The remaining policy order is merged with a heuristic tail sorted by instantaneous structural quality q_i, and at most $2k_t$ rows form expansion pool $\mathcal{R}_t$. For $S \subseteq \mathcal{R}_t$, define

$$\begin{aligned} F_t(S) = &\sum_{i \in S} (0.35 q_i + 0.20 r_i) + 0.25 \sum_{g \in \mathcal{G}} d_g \sqrt{\sum_{i \in S} p_{ig}} \\ &+ \frac{0.20}{|\mathcal{R}_t|} \sum_{u \in \mathcal{R}_t} \max_{i \in S} s_{ui}, \end{aligned} \tag{39}$$

In Eq. (39), t denotes a separation round, S the selected subset of expansion pool $\mathcal{R}_t$, i a selected cut, u a represented pool candidate, and g a variable role. The quantities q_i, r_i, d_g, s_{ui} denote structural quality, pointer-rank prior, role demand share, and clipped candidate similarity. The function $F_t(S)$ is surrogate set utility. The set A_t contains fixed policy anchors, $U \subseteq \mathcal{R}_t \setminus A_t$ is a completion subset, and $f(U)$ measures utility added to the anchors. These q_i, r_i, d_g are cut-selection quantities, not chamber cycles, resources, or action durations.

where r_i is the normalized pointer-rank prior, d_g is the demand share of role g in the expansion pool, and s_{ui} is clipped cosine similarity on generic-quality and role-mass descriptors. The score q_i combines efficacy, violation, integral support, recognized-role ratio, dominant-role ratio, and structural entropy. In Eq. (39), the first term retains instantaneous quality and the learned order; the square-root coverage term gives diminishing marginal gain to repeated role mass; and the final term makes the selected cuts represent the main structures in the pool. Starting from the anchors, the method recomputes the marginal gain of every unselected candidate and adds the largest until k_t cuts have been selected. With nonnegative weights, role masses, and clipped similarities, and $F_t(\varnothing) = 0$, the modular, concave-coverage, and facility-location terms form a monotone submodular objective. For fixed anchors A_t and expansion pool, define $f(U) = F_t(A_t \cup U) - F_t(A_t)$. Greedy completion under the cardinality constraint has the standard $1 - 1/e$ approximation property for this residual objective [37]. This ratio concerns coverage under the residual surrogate objective within the fixed pool; PDI and solution time are evaluated experimentally.

The final ordered sequence is denoted $S_t^\star$, with $\star$ labeling the output of this round. The k_t selected cuts are added to the current LP in anchor order followed by greedy-completion order. Candidate truncation, state encoding, count prediction, sequence decoding, and submodular completion jointly determine the ordered cut set used to strengthen the relaxation in the current round.

3.5. Policy training and exact solution

One time-limited SCIP solve forms a rollout, with its terminal PDI used as the cost. The current policy is executed in every eligible cut-selection round. Training uses the high-level fraction and low-level sampled sequence at the first valid candidate set of that rollout as its action sample, and the PDI of the complete solve is returned to both policies. Writing $R = -\text{PDI}$ as a reward, the low-level estimator is

$$\begin{aligned} \widehat{\nabla_\theta J} &= \frac{1}{B} \sum_{b=1}^{B} (R_b - \bar{R}_{\text{EMA}}) \nabla_\theta \log \pi_\theta(\tau_b), \\ \bar{R}_{\text{EMA}} &\leftarrow 0.9 \bar{R}_{\text{EMA}} + 0.1 \bar{R}. \end{aligned} \tag{40}$$

Algorithm 1: Cut selection by parallel hierarchical REINFORCE and beam search

Algorithm steps
Input: SCIP-validated rows C_t, high-level policy π_θ^H, low-level pointer policy π_θ^L, beam width $B = 3$, and the current SCIP cut limit.
Output: Ordered cut set $S_t^\star = (i_1, \ldots, i_{k_t})$ to be added to the current LP.
1. Train the policies. Two workers collect rollouts in parallel with terminal reward $R = -\text{PDI}$; a central learner applies hierarchical REINFORCE with separate EMA baselines and synchronous Adam updates. Fix the parameters for formal inference.
2. Form the pool. Sort by efficacy, break ties by original index, and retain at most 128 rows.
3. Encode the state. Compute 13 HEM features, ten role-aware features, and role masses; apply grouped projections, sigmoid-gated fusion, and LSTM encoding.
4. Determine the count. Predict the retained fraction with π_θ^H and obtain k_t after the 0.2 fraction cap, SCIP round limit, and 16-cut limit.
5. Initialize the beam. Set $\mathcal{B}_0$ to the empty sequence with log probability zero.
6. Expand. For $r = 1, \ldots, k_t$, obtain masked conditional probabilities from one glimpse plus pointer attention, extend every prefix in $\mathcal{B}_{r-1}$ by every unselected candidate, and accumulate log probability.
7. Prune. Retain the three highest joint-probability parent–candidate extensions as $\mathcal{B}_r$; after the loop, take the leading sequence in $\mathcal{B}_{k_t}$.
8. Fix anchors. Retain its first $\lceil 0.25 k_t \rceil$ rows as policy anchors A_t.
9. Form the expansion pool. Merge the remaining pointer order with structural-quality order, remove duplicates, and keep at most $2k_t$ rows as $\mathcal{R}_t$.
10. Complete submodularly. Set $S \leftarrow A_t$; while $\lvert S \rvert < k_t$, add the $i \in \mathcal{R}_t \setminus S$ maximizing $F_t(S \cup \{i\}) - F_t(S)$.
11. Add cuts. Add $S_t^\star$ to the current LP in anchor order followed by completion order.

The objective J is expected return, $\nabla_\theta J$ its parameter gradient, and a hat denotes a sample estimate. Here B is the rollout batch size and $b = 1, \ldots, B$ a rollout index. The return is R_b, τ_b is the cut sequence sampled at the first eligible candidate set of rollout b, and $\pi_\theta(\tau_b)$ its policy probability. The batch-mean return is $\bar{R}$ and the exponential-moving-average baseline is $\bar{R}_{\text{EMA}}$, with EMA a method label. These uses of B, b, τ_b differ from beam width, parent prefixes, and physical auxiliary identities.

Equation (40) gives the batch policy-gradient update formed from the first eligible candidate-set action sample and an exponential-moving-average baseline; B is the rollout batch and τ_b is the cut sequence sampled at the first eligible candidate set in rollout b. The equivalent loss is PDI-cost advantage times log probability. Low-level log probability is averaged over sequence length so that a longer cut sequence does not produce a larger gradient merely because it contains more terms. The high-level policy uses the same PDI cost and an independent EMA baseline and is updated every two epochs; both gradient norms are clipped at 2. Parallel environments collect rollouts, and a central learner updates both policies synchronously after each batch, forming a parallel hierarchical REINFORCE procedure [38, 39].

Training uses five instances and five random seeds. Each seed has two workers, each completing four 30-s/1024-MB rollouts per epoch, for eight rollouts per epoch and 60 epochs. Initial Adam learning rates are 10^{-4} for the low-level policy and 5×10^{-4} for the high-level policy and are multiplied by 0.96 every ten epochs. Training uses stochastic decoding and formal inference uses beam width three. Every configuration–seed combination uses the fixed epoch-60 parameters without benchmark- or test-set parameter selection. HEM-13D, feature-only 23D, and the complete 23D method are trained separately so that weights are never mixed across input semantics or completion environments.

RDMCT-HBS only orders and selects valid rows already generated by SCIP, and SPBS submits only solutions checked in the complete model. Neither changes the objective, constraints, integrality, or bound logic. The learned components therefore change the search path and finite-time bound convergence, whereas feasibility and optimality retain the same meaning as in default SCIP.

4. Computational experiments and results

The experiments address three manufacturing questions: how full occupancy and dummy use affect mode preference across lot sizes and cleaning conditions; how much waiting and cycle coordination can improve while preserving product makespan; and whether limited computation yields a valid schedule in time. Feasible-schedule availability and product makespan measure usability and batch efficiency; waiting and cadence measure timing quality; gap and PDI explain search behavior. Residency uniformity, compliance with real-time JIT demand, and closed-loop execution under hardware deviations are not measured by the existing experiments.

Cleaning-policy revision and scope of evidence.. The current model permits cleaning before the threshold, requires it at the threshold, and uses the expanded capacities in Eq. (5). Section 4.2 reports newly solved and validated results under this rule. Subsequent historical benchmarks retain the numerical results for their original rules, fixed layouts, and budgets, and are explicitly identified in headings and captions. Historical optimality certificates do not transfer directly to the expanded feasible region. New conclusions rely on schedules passing both original-model and independent physical checks; a time-out without an incumbent or an execution failure is not an infeasibility proof.

Revalidation under the current rule uses the joint model with clean/dirty-hand constraints. Historical experiments separately cover the homogeneous-hand fixed-layout benchmark and joint scheduling with hand eligibility, the latter in Section 4.9. Results under different rules, layouts, travel times, hand constraints, and resource budgets are reported separately and are not pooled for algorithm comparisons.

4.1. Experimental design and metrics

Historical experiments used a Windows 11 workstation with an Intel Core Ultra 7 265KF processor, 32 GB RAM, and an NVIDIA GeForce RTX 5060 Ti GPU with 8 GB device memory, with SCIP 10.0.1, PySCIPOpt 6.1.0, PyTorch 2.11.0, and Python 3.12.12. Revalidation under the current rule uses the same workstation and freezes the model, configuration, and policy checkpoints for each campaign; campaign records specify the applicable versions. Solve budgets and initialization costs are accounted for separately within each comparison family.

Historical benchmarks first evaluate mode and solver performance for prescribed structures under dual-source supply ($\sigma = 1$); Section 4.9 records historical joint-model algorithm comparisons, mode sweeps, source routes, and parameter variations. Historical evidence includes 960 paired SPBS runs, 1500 equal-budget refinement runs, 300 factorial runs, 80 one-factor perturbation runs, and 729 distribution-shift runs across

all methods on 40–64-product lots. These historical sample sizes are excluded from current-rule revalidation coverage. The experiments examine mode comparisons, waiting and cadence, feasible-schedule availability, bound convergence, and scale effects; the metrics below apply within each matched comparison family.

The compared methods use the same MILP, SCIP version, instances, and random seeds, and each comparison family uses common time and memory limits. Let $P(t)$ denote the primal bound of the minimization problem at time t, namely the objective value of the best incumbent, and let $D(t)$ denote the dual bound, namely the global lower bound certified by the search tree. Define $\Delta(t) = |P(t) - D(t)|$, $m(t) = \min\{|P(t)|, |D(t)|\}$, and $M(t) = \max\{|P(t)|, |D(t)|\}$. When the two bounds are finite, have the same sign, and differ, the terminal relative primal–dual gap is defined in SCIP as

$$G_{\text{end}} = 100\,\frac{\Delta(T)}{m(T)}. \tag{41}$$

SCIP sets $G_{\text{end}} = 0$ when the bounds coincide and returns $+\infty$ when they have opposite signs or valid bounds are not both available. Equation (41) normalizes by the smaller absolute bound and consequently does not cap G_{end} at 100%. A value above 100% indicates that the absolute bound difference exceeds the smaller absolute bound.

SCIP uses a different bounded normalization for the primal–dual integral. Define the instantaneous integral gap as

$$g_{\text{PDI}}(t) = 100\,\frac{\Delta(t)}{M(t)}, \qquad \text{PDI} = \int_0^T g_{\text{PDI}}(t)\,\mathrm{d}t. \tag{42}$$

Here $t \in [0, T]$ is elapsed solve time and T is the PDI horizon in seconds, distinct from the production-cycle capacity bound in Section 2. The functions $P(t), D(t)$ are the current primal upper and dual lower bounds; $\Delta(t)$ is their absolute difference and $m(t), M(t)$ the smaller and larger absolute bounds. The quantities G_{end} and $g_{\text{PDI}}(t)$ denote terminal relative gap and instantaneous integral gap. The subscripts end and PDI are metric labels; PDI has units of percentage points times seconds.

Equation (42) applies when the bounds are finite, have the same sign, and differ; SCIP sets $g_{\text{PDI}}(t) = 0$ for coincident bounds and $g_{\text{PDI}}(t) = 100$ otherwise. The instantaneous integral gap is therefore bounded to $[0, 100]$; lower PDI means that the bound gap closes earlier. PDI is zero only if the gap is zero for almost the entire timed horizon, whereas $G_{\text{end}} = 0$ only states that optimality is proved at termination. Both quantities follow SCIP definitions and use its solution statistics [32, 33].

Historical benchmarks use manufacturing instances as the inference unit: random repetitions are first aggregated within each instance, followed by nonparametric 95% bootstrap intervals and Wilcoxon signed-rank tests on paired instance differences, with Holm correction within prespecified analysis families; p denotes a test p-value. Current-rule revalidation reports only paired observations and within-configuration summaries, without significance inference. Time-limited runs remain in their respective summaries, and random seeds are not treated as independent manufacturing instances.

4.2. Revalidation with optional cleaning before the threshold

Protocol and coverage.. The revalidation comprises core validation A, mode sweep B, and component ablation C. Each configuration uses solver seeds 1, 2, and 3 and a 300-s main-solve budget; refinement allocates 270 s to the main solve and 30 s to the second stage. Shared SPBS precomputation has a separate 20-s budget. Per-process memory limits are 14000 MB for A and B and 6000 MB for C, so performance measures are not pooled across campaigns. A runs serially; B includes early serial runs and a foreground queue subject to memory checks; C permits at most two independent tasks and at most one learned-policy task on the same GPU. Comparisons first match configurations and seeds, then aggregate the three seeds within each configuration. Learned methods use the frozen training-seed-1, epoch-60 checkpoint for transfer evaluation on the revised model, without retraining under the new rule.

Table 3 reports acceptance status for all planned runs. Two fixed-structure runs in A failed the original-model check and remain failures; feasibility checks recovered afterwards are not substituted as successful observations. All 24 failures in B occurred when shared initialization reached the memory limit for 24-product pure 2×2 configurations at $H = 3$–10; these are not infeasibility proofs. Of the 192 valid schedules in B, 168 main solves stopped at the time limit and 24 at the memory limit. All 216 runs in C passed original-model and independent physical checks, with time-limited main solves. Sensitivity campaign D was unfinished at the evidence freeze and is excluded from these conclusions.

Table 3: Planned runs and acceptance status under the current cleaning rule.

Campaign	Planned	Opt.	Feas.	No inc.	Failed
A: core	126	4	108	12	2
B: modes	216	0	192	0	24
C: ablation	216	0	216	0	0

Optimality refers to the corresponding solve domain; all four optimal runs in A are fixed-structure subproblems. Feas.: feasible at a time or memory limit. No inc.: timeout without an incumbent.

Initialization and structural comparison.. Four matched configurations in A each use three seeds: four products with dual-source supply at $H = 3, 10$, four with single-source supply at $H = 3$, and eight with dual-source supply at $H = 3$. None of the 12 cold-start runs obtains a feasible solution within 300 s, whereas all 12 runs with serial initialization and all 12 with SPBS initialization return valid schedules. SPBS makespans are 857, 857, 1491, and 2069 s, respectively, versus 888, 888, 1520, and 2300 s with serial initialization, reductions of 3.49%, 3.49%, 1.91%, and 10.04%. This supports initialization for the tested configurations; offline construction costs are excluded, so it does not establish end-to-end acceleration.

For four products with dual-source supply and $H = 10$, valid seeds 1 and 3 prove an optimum of 532 s within the original fixed structure. Reassigning auxiliary identities by chamber while preserving product pairing, chamber assignments, and the cycle skeleton yields feasible makespans of 566, 410, and 410 s for seeds 1, 2, and 3. The seed-3 schedule improves on the

matched fixed-structure optimum by 122 s (22.93%), with 107 s of overlapping chamber production. This existence comparison demonstrates that the original auxiliary identities restrict the attainable objective. The 410-s value remains a feasible upper bound, neither a global joint optimum nor evidence that every seed improves. The corresponding full joint search retains its 857-s initial schedule within 300 s, illustrating that a larger structural search space does not guarantee a better time-limited solution.

Equal-budget refinement.. C covers eight mixed-recipe configurations combining four/eight products, $H = 3, 10$, and single-/dual-source supply, with nine methods and three seeds per configuration. Paired methods use a common initial solution with identical hashes. Table 4 compares 300-s direct solving with 270-s main solving plus 30-s refinement. Total waiting decreases by 22.29%–50.25%, while product makespan remains unchanged within solver tolerances of order 10^{-6} s. Entries aggregate the three seeds within each configuration; all three yield the same displayed metrics, rounded to whole seconds. Maximum individual waiting does not decrease in every configuration.

Table 4: Equal-budget refinement for eight configurations under the current rule (times in seconds).

N/H	Source	$C_{\max}$	Total wait	Drop (%)	Max. wait
4/10	Dual	857	681→400	41.26	140→71
4/10	Single	1491	1110→600	45.95	124→65
4/3	Dual	857	681→400	41.26	140→71
4/3	Single	1491	1110→600	45.95	124→65
8/10	Dual	1723	1135→882	22.29	183→183
8/10	Single	2985	2412→1200	50.25	124→65
8/3	Dual	2069	864→578	33.10	74→74
8/3	Single	3933	3182→1680	47.20	118→65

$C_{\max}$ is unchanged; arrows indicate before→after refinement.

None of these eight configurations has a cadence-evaluation center formed by three consecutive production cycles. Recorded zero cadence deviation therefore provides no evidence of cadence improvement. The complete second-stage objective, waiting-only objective, and quadratic-penalty objective yield identical displayed waiting metrics; these samples cannot distinguish the effects of the cadence term or penalty form. An optimal second-stage subproblem also does not imply a global optimum for the first-stage joint problem.

Limits of cut-selection and mode-ranking evidence.. All six methods with 300-s main solves in C retain the common initial makespans, averaging 1925.75 s over eight configurations, with no observed change in the principal structure. Table 5 reports PDI and custom cut-selector callbacks. Learned methods trigger callbacks in only 18 of their 24 runs. The complete method differs descriptively from HEM in PDI but is close to the greedy ablation and does not improve the primary objective. These observations do not establish superiority of the learned policy or structural components. The three refinement variants have only 270-s main stages, so their PDI is not directly compared with that of 300-s methods.

Table 5: Component comparison for 300-s main solves under the current rule.

Method	Mean PDI	Callbacks	Runs triggered
Default SCIP	29513.46	0	0/24
HEM	29728.40	57	18/24
Complete method	29317.35	64	18/24
Feature-only	29375.97	65	18/24
Greedy selection	29317.24	72	18/24
No structural component	29490.91	64	18/24

Each method has eight configurations and three seeds; PDI is in %·s. Callbacks refer only to custom selectors; zero for default SCIP does not imply absence of solver separation.

The 12 matched comparisons of optional and threshold-only cleaning in A yield identical makespans, so strict objective improvement from relaxing the cleaning rule remains unproved. Three valid schedules in B use pre-threshold cleaning, establishing only that the option is exercised. Across the 48 complete mode triplets for eight and 16 products, matched by configuration and seed, no mode has a feasible upper bound below every competitor's valid lower bound. For 24 products, valid pure 2×2 observations are missing. Thus, this campaign establishes neither a global mode preference nor that a missing mode performs worse.

4.3. *Historical benchmark: Process-mode selection driven by cleaning position and lot size*

The numerical values in Table 6 define the historical fixed-layout computational configuration. The product-return comparison uses equal cumulative nominal processing time: 180 s for one 4×1 exposure and 90+90 s for two 2×2 exposures. Recipe qualification is a prescribed input condition for this scheduling comparison.

Table 6 specifies the experimental parameters for the representative 16-wafer Gantt charts. Two rotating chambers share the VTR, while product and auxiliary wafers compete for robots, load locks, and chamber spaces along their respective routes; cleaning counts, rotation, and transfer times are represented in the same MILP. All displayed schedules satisfy the original-model constraints checked by SCIP and an independent event-time validation. The charts compare resource occupancy and event coordination between the two modes.

Steady-state analyses of multi-space CTs usually measure cycle time or throughput [4]. For finite batches, we define $C_{\max}$ as the time when the last product wafer returns to an LP and define equivalent batch output as WPH $= 3600|\mathcal{W}|/C_{\max}$. Pure 4×1 uses 180 s per wafer, pure 2×2 uses 90+90 s, and a 1:1 mixture assigns each product its mode-specific recipe. The cleaning threshold H counts full-occupancy production cycles per chamber: a 4×1 batch counts once and each 2×2 exposure counts separately. In this historical fixed-layout experiment, early cleaning after $H - 1$ cycles was allowed only to reserve capacity for the following complete 2×2 chain. That layout generator forms production segments using cycle limits and complete-chain boundaries, including 500-s cleaning and associated auxiliary loading, unloading, rotation, and transport.

Table 7 compares cleaning endpoints; Table 8 gives the full $H = 3$–10 grid. The $H = 4$–9 experiment contains 36 con-

Table 6: Historical benchmark: Experimental parameters for the cleaning-frequency mode experiment and representative Gantt charts under equal total nominal processing time.

Group	Parameter and value	Group	Parameter and value
Rotating chambers	CH1 and CH2 share VTR; 4 wafer positions per chamber	Recipe duration	4×1: 180 s; 2×2: 90+90 s
Cleaning	At most $H = 3$–10 production cycles between cleanings; auxiliary-only cleaning lasts 500 s	Chamber actions	180° rotation: 2 s; gap between adjacent chamber actions: 1 s
ATR	1 two-hand robot, capacity 2; load/unload 4 s; complete loaded transfer 11 s	VTR	1 four-hand robot, capacity 4; loaded transfer 4 s; empty repositioning 4 s
ATR details	LL–LP empty return 6 s; PW-pair exchange at AL 8 s	AL	Capacity 1; minimum alignment occupancy 20 s per wafer
Load locks	Upper-slot pumping/reset 30 s; lower-slot venting/reset 25 s	LL capacity	Upper and lower load locks each contain 2 independent slots
Wafer sources	LP1–LP4, each holding 25 wafers	Auxiliary wafers	8 reusable wafers, 4 per chamber; storage capacity 10 wafers
Batch and model	Basic product batch size 4; 13 route steps	Chamber bound	At most 15 4×1 batch slots per PM
Model bounds	Big-M = 10000; module and robot residency bounds 10000 s	Waiting option	Avoidable-wait hard cap 0 disables this optional restriction
Objective	Product-return makespan and waiting refinement	Policy	Training seed 1, epoch 60, 23-dimensional features
Decoder and cuts	Beam width 3; candidate pool 128; at most 16 cuts per round	Solver	SCIP seed 1; available, validated initial schedules
Solve and chart selection	Product-return objective; makespan bounds for mode comparison; waiting-refined schedules for Gantt charts	Schedule validation	Original-model and independent validation of every schedule

figurations with training and solver seeds fixed at 1, a 14400-s main-solve limit, and stage 2 disabled. All 36 schedules pass independent validation; 35 are proven optimal. For 16 wafers at $H = 5$, pure 2×2 reaches a feasible 907 s within that budget. A separate 28800-s solve of the same instance, initialized from this schedule, improves the upper bound to 898 s, with lower bound 694 s and terminal gap 29.39%. The improved schedule passes main-model and 4-s exchange checks. This extended-budget experiment determines the feasible upper bound used for mode comparison; its computation time is excluded from the fixed-budget algorithm comparison. With cleaning inactive, the 16-wafer pure 2×2 models at $H = 6$–10 are identical and have optimum 898 s.

For 8 wafers, the three makespans remain 610, 578, and 654 s throughout $H = 3$–10. Pure 2×2 is 5.25% shorter than pure 4×1. At $H = 3$, pure 2×2 and mixed operation generate two and one terminal cleans, respectively; no cleaning occurs at $H \geq 4$. No internal cleaning is required at this lot size, and the ranking remains unchanged across the tested intervals.

For 16 wafers, the preferred mode changes with the interval. At $H = 3$, pure 2×2 and mixed operation each contain two internal cleans, and pure 4×1 is preferred at 944 s. At $H = 4$, pure 2×2 still requires two internal cleans, while mixed operation has only two terminal cleans and improves on pure 4×1 at 939 s. At $H = 5$, pure 2×2 has only two terminal cleans and its feasible 898-s schedule is shorter than the proven optima of both competitors. Thus the preference between the pure modes switches from 4×1 at $H = 4$ to 2×2 at $H = 5$. At $H = 6$–10, all modes avoid cleaning; the pure 2×2 optimum of 898 s is 4.87% and 4.37% shorter than pure 4×1 and mixed operation, respectively.

Complete-chain segmentation and cleaning position jointly explain this change. Each chamber in the 16-wafer pure 2×2 configuration processes four product PWs. The layout splits these into 2+2 PWs at $H = 3$, 3+1 at $H = 4$, and one complete chain at $H \geq 5$. Although $H = 3$ and 4 have equal internal-cleaning counts, their supply and return orders differ; the latter layout yields 1470 s. Mode selection therefore depends on lot size, recipe composition, and admissible complete-chain length. The experiments reconstruct preset complete-chain layouts for each H, so the comparison reflects the joint effect of the cleaning limit and chain layout.

The 24-wafer extension covers all three modes at $H = 3$–6 with 7200 s per configuration. Pure 4×1 has optimum 1268 s at each interval. At $H = 4$, pure 2×2 and mixed-mode lower bounds are 1342 and 1774 s; at $H = 5$, the pure 2×2 lower bound is 1440 s and the mixed optimum is 1796 s. These bounds establish pure 4×1 as preferred at both intervals. At $H = 6$, the mixed-mode upper bound of 1263 s is below the pure 4×1 optimum and pure 2×2 lower bound of 1440 s, establishing mixed-mode dominance; its terminal gap is 10.40%. At $H = 3$, a 28800-s solve for pure 2×2 obtains a validated 2220-s schedule, lower bound 2082 s, and gap 6.63%. This lower bound and the mixed-mode lower bound of 2302 s exceed the pure 4×1 optimum. The extended-budget solve determines mode ordering; its computation time is excluded from the fixed-budget algorithm comparison. Hence pure 4×1 is preferred for 24 wafers at $H = 3$–5 and mixed operation at $H = 6$. The difference from the 16-wafer $H = 5$ result demonstrates the dependence on lot size and complete-chain layout.

All 12 configurations in the 24-wafer $H = 7$–10 extension yield validated schedules. An independent solve for mixed operation at $H = 10$ proves a makespan of 1250 s within the 28800-s budget. The mixed-mode MILPs at $H = 7$–10 have identical objectives, constraints, and variable bounds, so all four intervals have the same optimal makespan. Pure 2×2 yields a feasible 1237-s schedule at $H = 8$–10. Transferring its production-event timing to the $H = 7$ model and scheduling two terminal cleanings also gives a 1237-s schedule that passes validation against the original model. The valid pure 2×2 intervals are $[712, 1237]$ s at $H = 7$ and $[842, 1237]$ s at $H = 8$–10. These upper bounds are below the mixed-mode optimum of 1250 s and pure 4×1 optimum of 1268 s. Thus, pure 2×2 is preferred throughout $H = 7$–10, with the constructed schedules reducing

Table 7: Historical benchmark: Cleaning-endpoint comparison under equal cumulative nominal processing time. Cleaning counts sum internal and terminal cleans across both chambers. † marks a validated makespan upper bound; the corresponding WPH uses that makespan.

Lot (wafers)	H	Mode	Total cleans	$C_{\max}$ (s)	WPH (wafers/h)
8	10	Pure 4×1	0	610	47.21
8	10	Pure 2×2	0	**578**	**49.83**
8	10	4:4 mixed	0	654	44.04
8	3	Pure 4×1	0	610	47.21
8	3	Pure 2×2	2	**578**	**49.83**
8	3	4:4 mixed	1	654	44.04
16	10	Pure 4×1	0	944	61.02
16	10	Pure 2×2	0	**898**	**64.14**
16	10	8:8 mixed	0	939	61.34
16	3	Pure 4×1	0	**944**	**61.02**
16	3	Pure 2×2	4	1402	41.08
16	3	8:8 mixed	2	1530	37.65
24	10	Pure 4×1	0	1268	68.14
24	10	Pure 2×2	0	**1237**†	**69.85**
24	10	12:12 mixed	0	1250	69.12
24	3	Pure 4×1	2	**1268**	**68.14**
24	3	Pure 2×2	6	2220†	38.92
24	3	12:12 mixed	4	2324†	37.18

makespan by 1.04% and 2.44%, respectively. Extended-solve and schedule-transfer costs are accounted for separately from the fixed-budget algorithm comparison.

The four resource Gantt charts use 16 wafers and compare the pure modes at $H = 10$ and 3. To study waiting at minimum makespan, the $H = 10$ pure 2×2 schedule is further optimized for total route wait under $C_{\max} \leq 898 + 10^{-6}$ s, with resource order allowed to change. The minimum route wait is 492 s, accumulated over six consecutive-operation intervals for each product wafer: 448 s from AL pair coordination, 36 s before post-processing unloading, and 8 s before return from the lower load lock. This route measure is distinct from the interval between consecutive in-chamber exposures. Each time axis ends at the last product return, $C_{\max}$. Every pure 4×1 process lasts 180 s and every pure 2×2 exposure lasts 90 s.

Chamber lanes distinguish gray exchange actions from pale-pink waiting. A combined exchange by the four-finger VTR takes 4 s. A split exchange includes 4-s unloading, at least 4 s of empty repositioning, and 4-s loading; other wafer-handling actions may be coordinated between unloading and loading. In Fig. 6, the first CH1 inter-exposure interval is 63 s: 59 s waiting followed by a 4-s combined exchange. The next product PW, W5/W6, enters the upper load lock at 299 s and completes vacuum-side preparation at 329 s. CH1 finishes processing at 270 s, waits until 329 s, and exchanges at 329–333 s. Most adjacent cycles in the $H = 3$ short-chain schedule use continuous 4-s combined exchanges; CH1 cycles five and six are separated by waits at 1106–1108 s and 1112–1116 s, unloading at 1108–1112 s, and loading at 1116–1120 s. The final exposure starts immediately at 1120 s and ends at 1210 s. Every exposure remains 90 s. Constraining all eight inter-exposure intervals in the same $H = 10$ model to exactly 4 s yields an independently validated optimum of 960 s, 62 s (6.90%) above 898 s. Continuous exchanges compress local intervals but restrict supply and return order on shared resources; allowing local waiting shortens this finite batch's makespan.

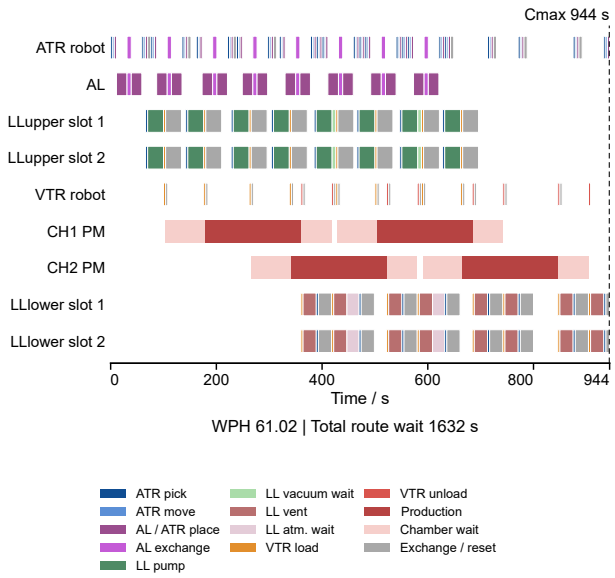


Figure 5: Historical benchmark: Homogeneous-hand, fixed-layout benchmark: 16-wafer pure 4×1 schedule at $H = 10$. No cleaning occurs; $C_{\max} = 944$ s, WPH is 61.02 wafers/h, and total route wait is 1632 s.

4.4. Historical benchmark: Equal-budget refinement of waiting and cadence

Waiting-time constraints can propagate to manufacturing-system output and control performance [21]. For the same 600-s total budget, the complete two-stage workflow assigns 540 s to the main solve and up to 60 s to refinement, whereas the one-stage control assigns all 600 s to the main solve. Figure 9 shows that the two-stage workflow reduces total route wait, maximum wait, cadence coefficient of variation, and cadence deviation by 51.91%, 68.89%, 39.70%, and 94.05%, respectively. All four paired instance differences remain significant after Holm correction, demonstrating that the two-stage workflow markedly improves waiting and cadence quality under the same total budget.

Table 8: Historical benchmark: Makespans, cleaning counts, and preferred modes under the prescribed PW layouts for 8-, 16-, and 24-wafer lots at $H = 3$–10. Times are in seconds.

Lot (wafers)	H	$C_{\max}$ (s)			$\Delta C_{\max}$ (s)	Total cleans			Preferred mode
		Pure 4 × 1	Pure 2 × 2	1:1 mixed		Pure 4 × 1	Pure 2 × 2	1:1 mixed	
8	3	610	578	654	-32	0	2	1	Pure 2 × 2
8	4	610	578	654	-32	0	0	0	Pure 2 × 2
8	5	610	578	654	-32	0	0	0	Pure 2 × 2
8	6	610	578	654	-32	0	0	0	Pure 2 × 2
8	7	610	578	654	-32	0	0	0	Pure 2 × 2
8	8	610	578	654	-32	0	0	0	Pure 2 × 2
8	9	610	578	654	-32	0	0	0	Pure 2 × 2
8	10	610	578	654	-32	0	0	0	Pure 2 × 2
16	3	944	1402	1530	458	0	4	2	Pure 4 × 1
16	4	944	1470	939	526	0	2	2	1:1 mixed
16	5	944	898†	939	-46	0	2	0	Pure 2 × 2
16	6	944	898	939	-46	0	0	0	Pure 2 × 2
16	7	944	898	939	-46	0	0	0	Pure 2 × 2
16	8	944	898	939	-46	0	0	0	Pure 2 × 2
16	9	944	898	939	-46	0	0	0	Pure 2 × 2
16	10	944	898	939	-46	0	0	0	Pure 2 × 2
24	3	1268	2220†	2324†	952	2	6	4	Pure 4 × 1
24	4	1268	2928†	1827†	1660	0	4	2	Pure 4 × 1
24	5	1268	1769†	1796	501	0	2	2	Pure 4 × 1
24	6	1268	1880†	1263†	612	0	2	1	1:1 mixed
24	7	1268	1237†	1250	-31	0	2	0	Pure 2 × 2
24	8	1268	1237†	1250	-31	0	0	0	Pure 2 × 2
24	9	1268	1237†	1250	-31	0	0	0	Pure 2 × 2
24	10	1268	1237†	1250	-31	0	0	0	Pure 2 × 2

Cleaning counts sum internal and terminal cleans across both chambers. † denotes a feasible upper bound; other times are proven optima. $\Delta C_{\max}$ subtracts the listed pure 4 × 1 time from the pure 2 × 2 time. A preferred mode is established by comparing optima or by a feasible upper bound below the competing lower bounds.

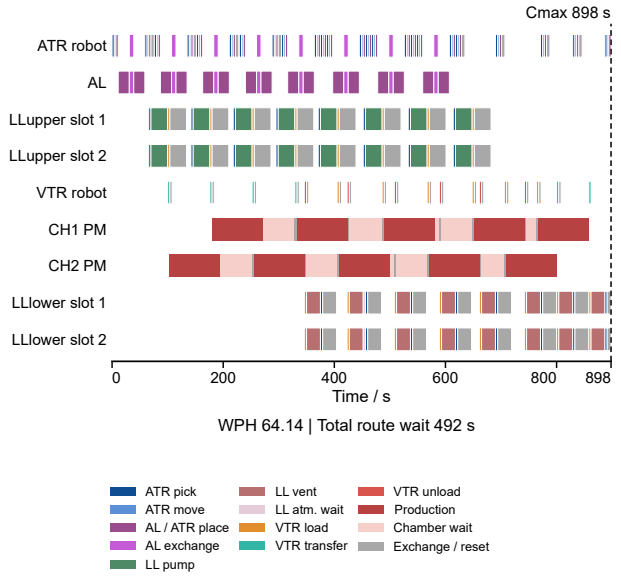


Figure 6: Historical benchmark: Homogeneous-hand, fixed-layout benchmark: 16-wafer pure 2 × 2 schedule at $H = 10$, without cleaning: $C_{\max} = 898$ s, WPH = 64.14 wafers/h. Refinement minimizes total route wait to 492 s at this makespan. Gray denotes 4-s chamber exchanges; pale pink denotes waiting.

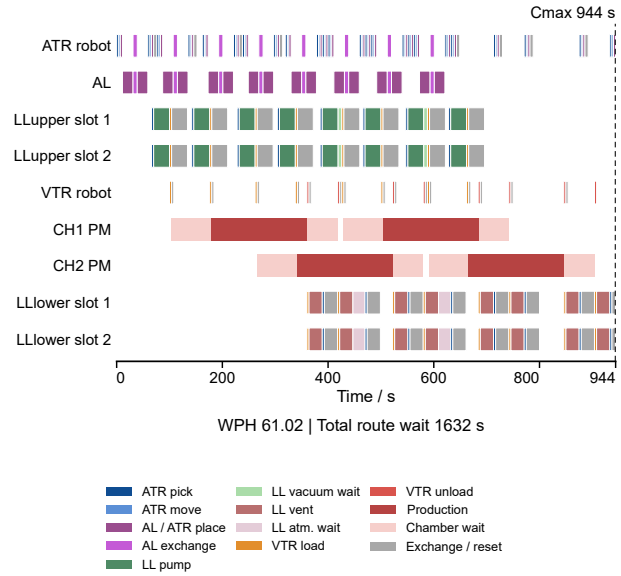


Figure 7: Historical benchmark: Homogeneous-hand, fixed-layout benchmark: 16-wafer pure 4 × 1 schedule at $H = 3$: $C_{\max} = 944$ s, WPH is 61.02 wafers/h, and total route wait is 1632 s.

The quadratic-objective control and the complete linear refinement both use the 540+up-to-60-s split. Linear refinement yields a cadence deviation of 22.04, 63.38% below the quadratic profile's 60.18, and the matched difference remains significant after Holm correction ($p = 0.0377$); total route wait, maximum wait, and cadence coefficient of variation are similar overall. The linear objective therefore reduces the measured cadence deviation under the same budget.

Table 9 isolates the second stage and refinement objective, complementing the initialization ablation in Table 10.

Table 9: Historical benchmark: Ablation of second-stage refinement and objective form. Each configuration contains 20 instances and 500 runs under a 600-s total budget. Entries are instance-aggregated means; lower is better.

Configuration	Total wait (s)	Max. wait (s)	Cadence CV	Deviation (s)
Full linear refinement	2596.01	296.33	0.130	22.04
Quadratic refinement	2469.47	293.27	0.134	60.18
Without stage two	5398.60	952.57	0.216	370.41

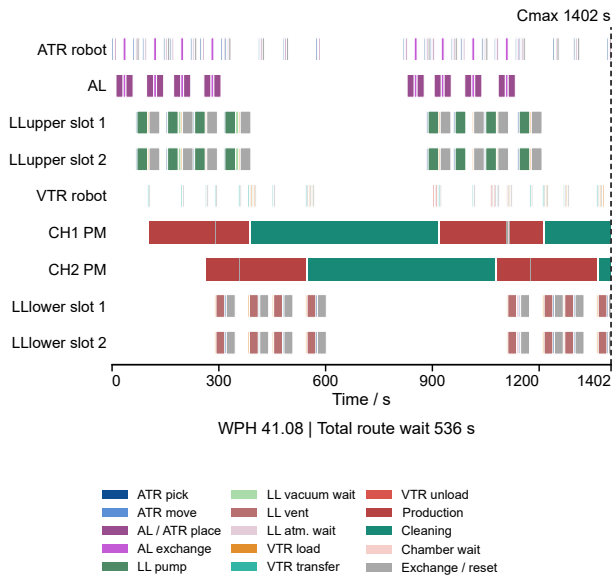


Figure 8: Historical benchmark: Homogeneous-hand, fixed-layout benchmark: 16-wafer pure 2×2 schedule at $H = 3$: two internal and two terminal cleans, $C_{\max} = 1402$ s, WPH = 41.08 wafers/h, and total route wait = 536 s. The horizon ends at product return; exchanges and waiting are shown separately.

4.5. Historical benchmark: SPBS and structure-aware cut selection

Table 10 summarizes the paired SPBS study with 48 instances and ten paired seeds per instance. The timed comparison measures main-solver performance after offline SPBS construction. Relative to the no-start condition, precomputed SPBS increases incumbent availability from 51.67% to 100%, a gain of 48.33 percentage points (95% interval [34.58, 62.29]). Mean PDI falls from 27473.05 to 20278.78, a decrease of 7194.27 or 26.19% (95% interval [5145.78, 9317.61]). Both paired differences remain significant after Holm correction ($p = 2.55 \times 10^{-5}$ and $p = 3.46 \times 10^{-8}$). Precomputed SPBS therefore improves feasible-schedule availability and primal–dual convergence during the subsequent solve.

Table 10: Historical benchmark: Effect of precomputed SPBS on time-limited MILP search; construction time is excluded. Each condition contains 48 instances and ten paired seeds per instance, totaling 480 runs.

Metric	No start	SPBS	Observed change
Incumbent rate (%)	51.67	100.00	+48.33 points
Mean PDI	27473.05	20278.78	-26.19%

HEM-13D is the most direct learning baseline for the structure-aware cut selector. Both methods share SPBS, training instances, random seeds, candidate limits, hidden dimensions, and a 600-s solve budget, with 500 runs per method. Table 11 presents this prespecified direct comparison. Relative to HEM-13D, RDMCT-HBS changes mean PDI from 16823.18 to 16585.70, mean solve time from 219.65 to 218.32 s, mean terminal gap from 127.24% to 126.01%, and proven-optimal rate from 62.4% to 63.2%. All 500 complete-method runs produce feasible schedules. After aggregating repeats over 20 manufacturing instances, all four paired mean-difference 95% bootstrap intervals include zero and all Holm-adjusted p-values are 1.0. The observed means are close under the common SPBS condition; the paired intervals span differences of either sign, and this comparison is therefore expressed in terms of observed means.

The comparison jointly evaluates role-aware representation, sequence decoding, and structural completion under common

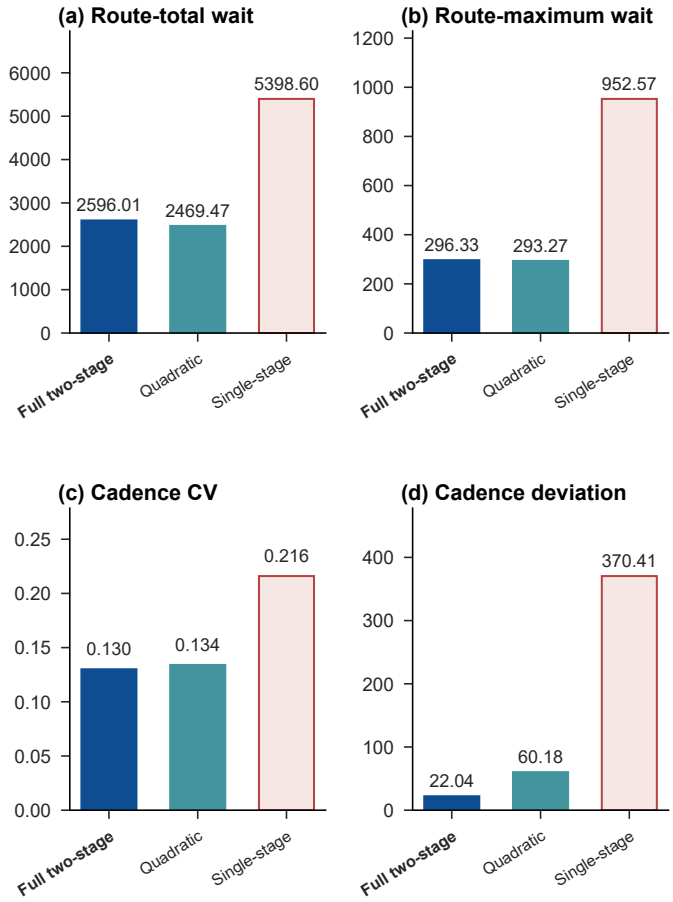


Figure 9: Historical benchmark: Final-schedule quality under the same 600-s total budget. a–d, total route wait, maximum route wait, cadence coefficient of variation, and cadence deviation; 1500 runs in total, with observed means labeled above the bars.

Table 11: Historical benchmark: Direct comparison of RDMCT-HBS and HEM-13D under a common SPBS and budget.

Metric	HEM-13D	RDMCT-HBS	Observed change
Mean PDI	16823.18	16585.70	-1.41%
Mean solution time (s)	219.65	218.32	-0.61%
Mean terminal gap (%)	127.24	126.01	-0.97%
Proven-optimal rate (%)	62.4	63.2	+0.80 points

initialization and solver budgets, quantifying their effects on PDI, solution time, terminal gap, and proven-optimal rate with shared SPBS.

4.6. *Historical benchmark: Scale, recipe composition, and computational difficulty*

The factorial design contains $4 \times 5 \times 3 = 60$ wafer-count–recipe-composition–process-scale cells with five solver seeds per cell, for 300 runs. Figure 10 shows a sharp transition. All 75 runs at both 8 and 16 wafers prove optimality, with mean PDI values of 28.13 and 1876.54 and mean solution times of 0.34 and 33.86 s. At 24 and 32 wafers, the proven-optimal rates fall to 36% and 20%, mean PDI rises to 31155.94 and 32959.89, and mean solution time reaches 408.55 and 433.20 s.

Table 12 summarizes the algebraic dimensions of the factorial instances. Each wafer count comprises 15 distinct parameter configurations; scalar variables and constraints are counted before presolve, without duplicating models across solver seeds. The mean variable counts are 2645.2 and 6711.2 for 16 and 24 wafers, respectively. Under the nominal process scale and the 1:1 mixed recipe, the corresponding counts are 2722 and 6630. These totals include layout-fixed indicators and linearization auxiliaries and thus characterize the algebraic representation; independent decisions also depend on fixing relations and process constraints.

Algebraic representation size and the number of candidate sequences follow different growth rates. For K_r candidate operations on resource r, pairwise ordering requires at most $\binom{K_r}{2}$ binary variables, giving an upper bound of $\sum_{r\in\mathcal{R}} \binom{K_r}{2}$ across resources. Before precedence restrictions are imposed, a single resource admits $K_r!$ permutations. Process precedence and resource coupling further restrict these candidate orders. Thus, a quadratic ordering representation can encode factorially many candidate sequences, and variable dimension and combinatorial search size should be assessed separately.

The prespecified PDI factorial model has coefficient of determination $R^2 = 0.998$. Under the 1:1 mixed recipe, the additional interaction coefficients at 24 and 32 wafers are 37729.48 and 40788.63, while the 95% intervals for the process-scale main effect and all prespecified process-scale interactions include zero. Over the tested 0.9–1.1 process-scale range, observed differences in computational difficulty are primarily associated with the interaction between batch size and mixed-flow composition, with comparatively small differences associated with process scale.

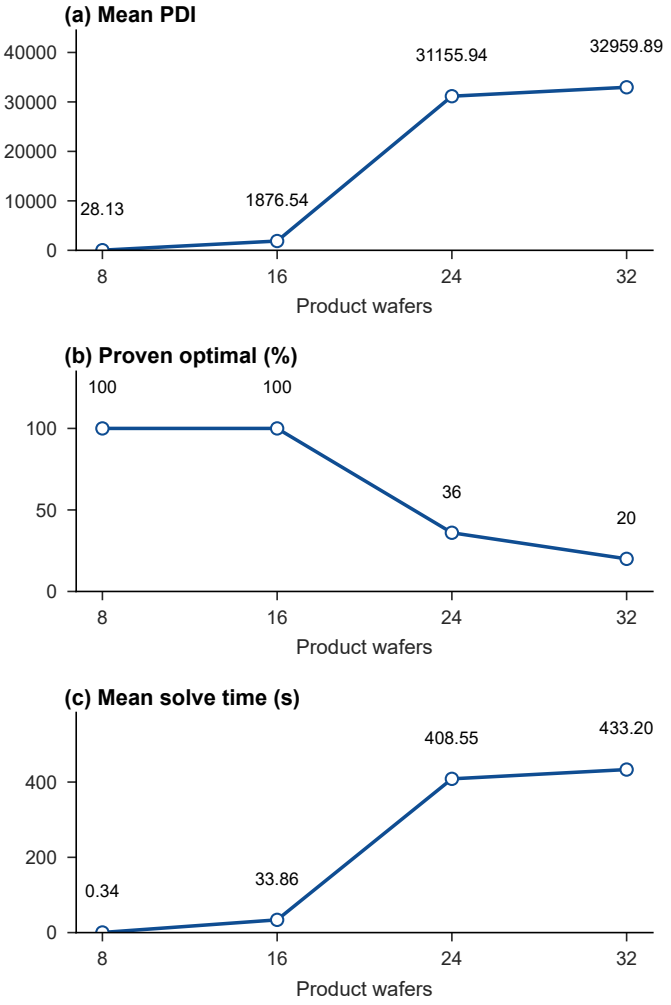


Figure 10: Historical benchmark: Scale transition in the 300-run factorial experiment. a, mean PDI; b, proven-optimal rate; c, mean solution time. Each wafer count contains 75 runs and observed values are labeled directly.

4.7. *Historical benchmark: Operation under parameter perturbations*

Prior CT studies show that cleaning actions and their triggering conditions alter robot-task sequences, cycle time, and wafer delay [16, 18]. The one-factor sensitivity study therefore covers eight prescribed perturbations of transport, processing, cleaning threshold, and vacuum-domain auxiliary inventory, for 80 runs. Every run returns a solver-checked feasible schedule and 20 runs prove optimality; scenario mean PDI ranges from 28899.41 to 42478.32. A longer cleaning threshold gives the lowest mean PDI and slower transport the highest, describing the variation in search performance across the tested parameter settings.

4.8. *Historical benchmark: Feasible-schedule delivery under distribution shift*

The distribution-shift suite enlarges batches to 40–64 wafers and changes mixed-flow composition, producing 729 runs across all compared methods over nine shifted manufacturing scenarios, including 135 runs of the complete method. This

Table 12: Historical benchmark: Model dimensions in the factorial experiment with prescribed PW layouts. Each wafer count comprises 15 models. Entries are means except for the variable range; all models have zero general integer variables.

Wafers	Variables	Binary	Continuous	Variable range	Constraints
8	1163.6	644.6	519.0	892–1450	2947.0
16	2645.2	1632.4	1012.8	2217–3457	8727.0
24	6711.2	4962.4	1748.8	5507–9362	23670.8
32	9984.0	7547.2	2436.8	7801–13663	41630.8

experiment evaluates schedule availability and search performance under changes in scale and composition. All 729 runs return an incumbent that passes SCIP and independent post-solve checks. The shared SPBS–MILP–SCIP chain therefore continues to output a validated resource-action schedule over the tested changes in scale and recipe composition. The proven-optimal rate is 0%. The complete method has mean PDI 73919.15, compared with 73670.99 for default SCIP and 70915.54 for adaptive cut scoring. Feasible-schedule availability is 100% across this suite, while adaptive cut scoring has the lowest observed mean PDI.

4.9. Historical benchmark: Joint scheduling with clean and dirty VTR hands

Six supplementary configurations cover single- and dual-source six-wafer lots at $H = 3$, dual-source eight-wafer lots at $H = 3$ and 10, and dual-source 16- and 24-wafer lots at $H = 3$. Single-source auxiliary wafers travel between atmospheric storage and the chambers via ATR and the load locks without alignment; dual-source auxiliary wafers use vacuum storage and VTR. Each lot contains equal numbers of the two product recipes, distributed successively across LP1–LP4; same-recipe products may be paired across load ports. Eight physical auxiliary wafers form a shared inventory. VTR pickup, travel, and placement take 1, 2, and 1 s, respectively, and rotation takes 2 s. No additional travel is imposed between upper and lower slots at the same load-lock interface. Processing takes 180 s or two 90-s exposures. The fixed policy assigns same-recipe pairs to both chambers according to nominal workload; the joint model also selects membership, chamber assignment, and cycle occupancy. Both formulations optimize resource orders and event times. The VTR uses two dirty hands for unprocessed products and two clean hands for fully processed products; auxiliary wafers may use any free hand. Each complete holding interval occupies one unchanged physical hand.

All five settings within each configuration use exactly the same initial variable assignment, with solver and policy-training seeds fixed at 1. Initial construction and auxiliary-variable completion are timed separately. The fixed policy, default SCIP, HEM, and the complete selector each receive a 300-s main-solve budget; refinement receives 270 s for the main solve and up to 30 s for the second stage. All six refinement configurations use default SCIP and are compared with its 300-s one-stage results. Learned policies retain the frozen epoch-60 parameters trained on the fixed-layout benchmark. All 30 schedules pass independent checks of complete wafer routes, exposure counts, resource capacities, cleaning, and auxiliary reuse. Feasible makespans in the common-initialization comparison are measured by the actual time of the final product return to its original load port.

The fixed policy obtains shorter feasible makespans in four six- or eight-wafer configurations and ties the joint methods for 16 and 24 wafers. All 18 joint primary runs with default SCIP, HEM, and the complete selector retain the common initial pairing, auxiliary identities, chamber assignments, and cycle structure; product makespans also equal their initial values. Initialization supplies the feasible structures in this comparison, and the subsequent timing-refinement gains are given in Table 15. Learned selection is called once in each six- and eight-wafer configuration and zero times in the 16- and 24-wafer configurations. All three joint methods return the same structures and makespans within the tested budget. Differences between fixed and joint settings describe time-limited search performance; the joint feasible region contains the corresponding fixed-structure subset.

Four- and six-product dual-source mixed instances further examine the relationship between fixed-structure and joint search. The two recipes have product counts of 2:2 and 3:3, respectively; both instances use $H = 10$, eight shared auxiliary wafers, and full candidate bounds. The four-product instance places one 4×1 product at each of LP1 and LP2 and one 2×2 product at each of LP3 and LP4. The six-product instance places one product of each recipe at both LP1 and LP2, with one additional 4×1 product at LP3 and one 2×2 product at LP4. The two settings for each instance share a complete initial variable assignment, use default SCIP with seed 1, and allow up to 1800 s for the primary solve with no second stage. Initialization takes 0.296 and 0.781 s, respectively, outside the main-solve budget. Table 13 gives both terminal lower bounds and feasible upper bounds. All four schedules pass complete-route, individual-hand, and auxiliary-reuse checks.

The four-product fixed structure has an optimal makespan of 532 s, 40.09% below the common initial schedule. The six-product fixed structure yields a feasible makespan of 957 s, a reduction of 37.45%. Both joint solves retain the initial pairing, auxiliary identities, chamber assignments, and cycle occupancy; the four-product improvement from 888 to 717 s arises from timing and resource coordination within that structure. The complete assignments for the 532- and 957-s fixed-structure schedules also satisfy the unrestricted joint model. These schedules therefore supply valid feasible upper bounds for the joint problems, while Table 13 preserves the bounds obtained by each independent search. The joint solves process 12 and 3 nodes, respectively, indicating slow search progress for

Table 13: Historical benchmark: Small-instance structural comparison for dual-source mixed lots ($H = 10$; times in seconds). Both settings allow up to 1800 s per primary solve; the four-product fixed structure closes the bound gap in 45 s.

N	Initial	Fixed structure		Joint model	
		Lower	Upper	Lower	Upper
4	888	532	532	46	717
6	1530	725	957	39	1530

this formulation after structural decisions are released within the tested budget.

The selected structures in both instances repeatedly borrow the same auxiliary wafers across the two chambers: every productive cycle contains D1 and D2. A physical auxiliary must be returned before its next loan, forcing the corresponding chamber exposures to occur sequentially. Balanced nominal processing loads therefore do not automatically produce chamber concurrency; auxiliary identities and complete loan intervals constrain the achievable overlap. None of these four schedules includes cleaning, so this comparison examines the coupling among auxiliary reuse, clean/dirty-hand allocation, and transport timing. Total auxiliary inventory, actual loan assignments, and the resulting event times must consequently be evaluated together.

Further structural checks initialize the complete joint model with the 532- and 957-s fixed-structure schedules, retaining all pairing, auxiliary-identity, chamber-assignment, and cycle decisions. Each run receives an additional 1800 s with seed 1 and no second stage. Both retain the supplied principal structure and makespan, with terminal lower bounds of 46 and 39 s, respectively. Table 14 presents these strong-initial-solution checks and a controlled auxiliary-identity comparison. Additional search time is accounted for separately from the preceding solves that produce the initial schedules.

The auxiliary-identity comparison preserves the four-product pairing, chamber assignments, cycle types, occupancy, and front/back structure, using the same shared inventory of eight auxiliary wafers. CH1 uses D1/D2 and CH2 uses D3/D4. This comparison fixes the revised auxiliary-loan structure while optimizing resource order, physical-hand assignments, and event times. Starting from a 628-s schedule obtained by a 60-s preliminary search, an additional 1800-s primary solve yields a feasible makespan of 403 s and a terminal lower bound of 393 s. The complete assignment also satisfies the unrestricted joint model, supplying a feasible upper bound of 403 s for the four-product joint problem.

In the revised schedule, CH1 processes in 4×1 mode during 173–353 s, while CH2 executes its two 2×2 exposures during 104–194 s and 204–294 s, producing 111 s of concurrent processing. The four products return to their original LPs at 403, 396, 347, and 340 s. Complete routes, exposure counts, individual-hand occupancy, and auxiliary reuse satisfy all constraints; no cleaning is required. The 403-s makespan is 129 s (24.25%) below the proven optimum of 532 s under the original auxiliary-loan structure. Because every timing arrangement within that original structure is bounded below by 532 s, the strict improvement establishes that revised auxiliary identities

Table 14: Historical benchmark: Strong-initial-solution and auxiliary-identity checks ($H = 10$, dual-source; times in seconds). Additional solves receive 1800 s each; the original fixed result is from Table 13. The final-row lower bound applies to the revised fixed auxiliary structure. Overlap is the total intersection of chamber-processing intervals. Preceding construction and solves are timed separately.

N	Setting	Initial	Lower	Upper	Overlap
4	Original fixed	888	532	532	0
4	Joint, strong start	532	46	532	0
6	Joint, strong start	957	39	957	0
4	Separate auxiliary identities	628	393	403	111

combined with timing reoptimization can outperform all schedules retaining the original structure. The comparison quantifies the value of auxiliary-loan structure at unchanged inventory. The common-initial-solution algorithm comparison remains as given in the preceding common-initialization comparison under the 300-s total solve budget.

Refinement sums transport-route waiting for both products and auxiliary wafers, subtracting required travel, alignment, or pressure-transition durations from successive handling and supply/return intervals. This measure is evaluated separately from the product-route waiting used in the preceding fixed-layout experiments. Table 15 shows reductions of 8.40%–29.21% in total transport-route waiting across the six configurations, with maximum individual waiting decreasing from 73 to 65 s in every case. For the six-wafer single-source configuration, total waiting decreases from 2200 to 1680 s, a reduction of 23.64%, and actual product makespan decreases from 3917 to 3875 s. Actual makespans also decrease in all five dual-source configurations. Under the common 300-s total solve budget, continuous-time refinement thus reduces both transport waiting and product makespan in the tested configurations.

The dual-source joint-model sweep covers $N = 8, 16, 24$ and $H = 3, \ldots, 10$, with pure 4×1, pure 2×2, and an equal-product-count mixture at each configuration. Each cell receives up to 20 s of hierarchical initialization and a 300-s main solve. Table 16 gives 72 independently validated feasible makespans, comparing schedules obtained under this budget and initialization strategy.

Among the schedules obtained, pure 2×2 gives the shortest eight-wafer makespan at $H = 5, 7$, whereas mixed operation gives the shortest at $H = 8$–10. For 16 wafers, the smallest observed makespans come from pure 4×1 at $H = 3$–6, mixed operation at $H = 7, 8$, and pure 2×2 at $H = 9, 10$. For 24 wafers, pure 4×1 gives the shortest makespan at every tested H, decreasing from 4640 s at $H = 3$ to 3945 s at $H = 6$–10, a reduction of 14.98%. These differences characterize the combined effects of cleaning conditions, lot size, and finite-budget search on the schedules obtained.

To examine both source routes, physical hands are assigned to eight existing schedules with two or four mixed-recipe products and $H = 2, 3$. The original schedules used homogeneous hands, a 120-s main-solve budget, and up to 20 s of hierarchical initialization. All eight satisfy clean/dirty-hand eligibility without changes to event times. Table 17 gives the recertified makespans. The two-product instances use different recipes and

Table 15: Historical benchmark: Equal-total-budget transport-wait refinement with clean/dirty VTR hands (s). Superscripts (1)/(2) denote 300-s one-stage and 270+up to 30-s two-stage solving. All rows use default SCIP. Makespans use actual product return times.

N	Source	H	$L^{(1)}_{\mathrm{sum}}$	$L^{(2)}_{\mathrm{sum}}$	Gain (%)	$C^{(1)}_{\max} \to C^{(2)}_{\max}$
6	Dual	3	356	252	29.21	2051 → 1916.91
6	Single	3	2200	1680	23.64	3917 → 3875
8	Dual	3	532	480	9.77	2300 → 2150.96
8	Dual	10	524	480	8.40	1779 → 1732.96
16	Dual	3	1100	960	12.73	4221 → 3886.80
24	Dual	3	1624	1440	11.33	6823 → 6066.30

Table 16: Historical benchmark: Feasible makespans for the clean/dirty-hand joint model under a 300-s main-solve budget (s). Mixed operation uses equal product counts of both recipes.

N	H	4×1	2×2	Mixed
8	3	1315	1658	1934
8	4	1315	1798	1315
8	5	1315	1230	1264
8	6	1315	1318	1315
8	7	1315	1267	1315
8	8	1315	1267	1227
8	9	1315	1267	1227
8	10	1315	1281	1227
16	3	2975	4716	3423
16	4	2633	3470	3056
16	5	2633	3056	2955
16	6	2633	2976	3055
16	7	2633	2874	2599
16	8	2633	3064	2585
16	9	2633	2554	2585
16	10	2633	2591	2636
24	3	4640	7336	5991
24	4	4361	6060	5359
24	5	4358	5434	5473
24	6	3945	5434	4835
24	7	3945	4796	4835
24	8	3945	4799	4835
24	9	3945	4799	4838
24	10	3945	4796	4203

pair each product with a auxiliary; the four-product instances pair same-recipe products across LP1 and LP2 and across LP3 and LP4. Products pass through AL and the load locks. Single-source auxiliary wafers travel directly from atmospheric storage to LL through ATR, whereas dual-source auxiliary wafers use vacuum supply and return. All complete routes, exposure counts, and shared-inventory constraints are satisfied. Dual-source schedules have shorter feasible makespans in all four configurations.

Processing-time and auxiliary-inventory variations are further evaluated with eight mixed-recipe products, $H = 3$, and dual-source supply, using 300 s for the main solve and 20 s for initialization. Table 18 covers ±10% changes in nominal processing times and inventories of four, six, and ten auxiliary wafers. All five cells yield complete feasible schedules, establishing schedule availability under these input changes for the modeled pairing, cleaning, and transport relations. The makespans are obtained by separate searches for the respective inputs.

Table 19 gives formulation dimensions before presolve for the two pure modes with 16 products and $H = 10$. Physical-hand allocation introduces both hand selection and exclusion between holding intervals on the same hand: the respective hand-assignment counts are 672 and 992, and hand-order

Table 17: Historical benchmark: Makespans of small single- and dual-source schedules recertified with clean/dirty hands (s). Single-source auxiliary wafers bypass alignment.

N	H	Single source	Dual source
2	2	2378	1028
2	3	1433	514
4	2	2436	1280
4	3	1491	854

Table 18: Historical benchmark: Parameter-variation results for the clean/dirty-hand joint model. Times are in seconds; processing durations refer to one exposure in each mode.

4×1 duration	2×2 duration	Auxiliary wafers	$C_{\max}$
162	81	8	1880
198	99	8	1988
180	90	4	1875
180	90	6	1888
180	90	10	1937

counts are 14028 and 30628. Chamber order is represented through cycle occupancy, first/last exposure, and front/back states. These counts include activation conditions, symmetric choices, and precedences implied by routes and thus measure algebraic representation size; feasible combinations are further restricted by wafer, capacity, and timing constraints.

For $H = 10$, dual-source supply, eight auxiliary wafers, and equal recipe counts, the complete 16- and 24-wafer models contain 124048 and 296700 variables, respectively. Considering only product pairing and assignment to two labeled chambers, even recipe counts give $N_\zeta!N_\psi!/[(N_\zeta/2)!(N_\psi/2)!]$ candidates: 2822400 for 16 wafers and 442597478400 for 24. This count includes label symmetry and precedes full-occupancy, cleaning, and routing constraints. It measures growth of a specified candidate class, rather than feasible schedules or independent variable dimensions.

4.10. Historical benchmark: Search-stage diagnostics with strengthened bounds

For the dual-source mixed configurations with 8 and 16 products and $H = 3$, default SCIP and the complete selector are each run for 120 s with a common feasible initial schedule and identical strengthened bounds. These comprise analytical lower bounds from mandatory routes and resource occupancy and a makespan upper bound from the initial schedule. The four diagnostic runs are separate from the preceding 300-s comparisons and identify the stages consuming the search budget.

For eight products, both methods retain a feasible makespan of 2300 s and terminate with a lower bound of 492 s. Both enter the first root LP solve, executing 25630 and 24675 LP iterations, respectively, before any learned cut-selection call. For 16 products, both retain a feasible makespan of 4221 s and spend the 120-s budget in presolve, with zero processed nodes and zero LP solves. All four final schedules pass independent physical validation. These observations locate the computational effort in presolve and root relaxation, identifying the large discrete formulation and continuous-relaxation solution as the initial search costs for these joint instances.

Table 19: Historical benchmark: Size audit of the 16-product, $H = 10$ joint models; counts refer to candidates before presolve.

Variable family	4×1	2×2
ATR order	4560	4560
VTR action order	27730	63190
VTR hand assignment	672	992
VTR hand-occupancy order	14028	30628
AL order	120	120
Upper-LL order	120	120
Lower-LL order	120	120
Auxiliary-loan order	2244	5724
All variables	78351	180925
All constraints	376764	887126

4.11. Discussion of mode selection and computational performance

Conditional process-mode selection.. Full occupancy makes dummy supply and return part of production organization, requiring mode comparisons to include filling, complete chains, and cleaning occupancy. In the historical fixed-layout benchmark for 16 products, pure 4×1 is preferred at $H = 3$ with 944 s, the mixture at $H = 4$ with 939 s, and pure 2×2 at $H = 6$ with 898 s. The decision concerns system makespan under specified conditions, rather than exposure duration alone. These findings apply to the original cleaning rule and threshold-dependent prescribed chain layouts. The same comparison principle applies to the current joint model, but insufficient bound separation prevents transferring historical switching points into universal rules for that model.

Current-rule experiments support three distinct conclusions: common initial solutions help the tested time-limited runs obtain feasible schedules; timing refinement after fixing the principal structure reduces waiting; and an alternative auxiliary-identity structure admits a feasible schedule strictly better than the original fixed-structure optimum. The 410-s structural example, waiting reductions from refinement, and feasible-schedule availability from initialization answer different questions and cannot be combined into a claim of superiority for the complete joint algorithm.

Expanding cleaning choices includes the choices permitted by the original rule at the model level, but time-limited heuristic search does not guarantee a better objective. Current matched runs do not show a makespan improvement from early cleaning, and mode-sweep lower bounds do not establish a preferred mode. Historical fixed-layout rankings established by optima or separated bounds apply only to their original rules and equipment settings.

Stable coordination and limits of engineering applicability.. Equal-budget refinement demonstrates reduced waiting within a selected structure; it does not establish reduced residency variance or strict JIT operation. Both hardware-deviation mechanisms motivate updates as parameters and states change, but a deterministic feasible solution obtained within a time limit does not establish execution feasibility under deviations. Deployment would additionally require measured duration and resource differences, state carryover at disruptions, rescheduling latency, and validation of execution constraints. These remain future validation tasks rather than completed results.

Learned cut selection under the current rule does not improve the common initial primary objective; some runs never invoke the custom selector, and callback counts are limited. The eight refinement configurations have no valid cadence-evaluation centers, so zero deviation cannot support cadence improvement. Further validation requires configurations with multiple production segments, complete mode coverage, and matched resource budgets. Unfinished sensitivity experiments remain pending until acceptance.

5. Conclusions

Starting from dummy occupancy required by full-occupancy processing, this study addresses conditional process-mode comparison and limited-budget scheduling in PECVD rotating-chamber cluster tools. The joint model connects product pairing, dummy loans, chamber assignment, complete chains, and cleaning boundaries within one physical timeline to evaluate two prescribed modes and their mixtures. Historical fixed-layout results for 16 products show preference switching between both pure modes and mixed operation as the cleaning threshold changes. Mode selection must therefore consider lot size, cleaning location, and resource coordination together; these switching results remain specific to the historical configurations.

Under the current rule, core validation, mode sweeping, and component ablation accept 112/126, 192/216, and 216/216 valid schedules, respectively. Across four matched core configurations, all 12 cold-start runs time out without a feasible solution, whereas all 12 serial-initialized and all 12 SPBS-initialized runs obtain feasible schedules. Equal-budget refinement over eight configurations with three seeds each preserves product makespan and reduces total route waiting by 22.29%–50.25%. The four-product dual-source auxiliary-identity comparison yields a feasible 410-s schedule below the proven 532-s optimum of the original fixed structure, showing that auxiliary-loan relationships materially affect attainable makespan.

The two stages separate product-batch efficiency from operating coordination: product-return makespan is optimized under residency and other physical constraints, then its bound is preserved while waiting and cycle-interval variation are reduced. This priority defines efficiency and timing quality explicitly, but does not directly optimize residency uniformity. Obtaining and validating a feasible schedule underpins the algorithm, while the contribution of learned search must be established under matched budgets. Action-duration deviations and resource-performance differences motivate further work on parameter updates and online scheduling.

These findings do not establish global optimality of the full joint problem, a necessary makespan reduction from early cleaning, or superiority of learned cut selection over baselines. The mode sweep has initialization memory failures and overlapping bounds in completed comparisons; refinement samples lack cadence-evaluation centers, and sensitivity experiments remain unfinished. Historical fixed-layout and original-rule results are retained separately and cannot substitute for evidence on these pending claims. Future work must complete mode

coverage, include configurations with valid cadence structure, and retrain and compare policies under the current rule with matched settings.

CRediT authorship contribution statement

Lincong Wei was responsible for Conceptualization, Methodology, Software, Validation, Formal analysis, Investigation, Data curation, Visualization, Writing – original draft, and Writing – review and editing. Lizhen Shao was responsible for Conceptualization, Methodology, Formal analysis, Validation, Writing – review and editing, and Project administration. Wei Wang was responsible for Formal analysis, Investigation, Writing – review and editing, and Supervision. All authors reviewed and approved the final manuscript.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data and research-material availability

The data, mathematical instances, trained models, and solver software supporting the findings are available from the corresponding author on reasonable request.

Declaration of generative AI and AI-assisted technologies in the manuscript preparation process

During the preparation of this work, the authors used OpenAI ChatGPT and Codex to assist with language editing, Chinese–English translation, LaTeX consistency checks, the development and debugging of numerical solution methods, and experimental visualization. After using these tools, the authors reviewed and edited the content as needed and take full responsibility for the content of the publication.

References

[1] Pan CR, Zhou MC, Qiao Y, Wu NQ. Scheduling cluster tools in semiconductor manufacturing: Recent advances and challenges. IEEE Trans Autom Sci Eng 2018;15:586–601. https://doi.org/10.1109/TASE.2016.2642997.

[2] Applied Materials. Producer XP Precision CVD. n.d. https://www.appliedmaterials.com/us/en/product-library/producer-xp-precision-cvd.html (accessed September 7, 2026).

[3] Gu L, Wu N, Qiao Y, Zhang S, Li T. Scheduling cluster tools with multi-space process modules and a multi-finger-arm robot in wafer fabrication subject to wafer residency time constraints. Appl Sci 2024;14:9490. https://doi.org/10.3390/app14209490.

[4] Gu L, Wu N, Qiao Y, Zhang S. Scheduling of Multi-finger-robotic Cluster Tools with Multi-space Process Modules. In: 2024 International Conference on Networking, Sensing and Control (ICNSC). IEEE; 2024. https://doi.org/10.1109/ICNSC62968.2024.10760109.

[5] Maitra V, Su Y, Shi J. Virtual metrology in semiconductor manufacturing: Current status and future prospects. Expert Syst Appl 2024;249:123559. https://doi.org/10.1016/j.eswa.2024.123559.

[6] Choi JE, Hong SJ. Machine learning-based virtual metrology on film thickness in amorphous carbon layer deposition process. Meas Sens 2021;16:100046. https://doi.org/10.1016/j.measen.2021.100046.

[7] Perkinson TL, McLarty PK, Gyurcsik RS, Cavin RK. Single-wafer cluster tool performance: An analysis of throughput. IEEE Trans Semicond Manuf 1994;7:369–73. https://doi.org/10.1109/66.311340.

[8] Rostami S, Hamidzadeh B. Optimal scheduling techniques for cluster tools with process-module and transport-module residency constraints. IEEE Trans Semicond Manuf 2002;15:341–9. https://doi.org/10.1109/TSM.2002.801379.

[9] Wu NQ, Zhou MC. A closed-form solution for schedulability and optimal scheduling of dual-arm cluster tools with wafer residency time constraint based on steady schedule analysis. IEEE Trans Autom Sci Eng 2010;7:303–15. https://doi.org/10.1109/TASE.2008.2008633.

[10] Qiao Y, Wu NQ, Zhou MC. Petri net modeling and wafer sojourn time analysis of single-arm cluster tools with residency time constraints and activity time variation. IEEE Trans Semicond Manuf 2012;25:432–46. https://doi.org/10.1109/TSM.2012.2199338.

[11] Qiao Y, Wu N, Zhou MC. Scheduling of dual-arm cluster tools with wafer revisiting and residency time constraints. IEEE Trans Ind Inform 2014;10:286–300. https://doi.org/10.1109/TII.2013.2272702.

[12] Wang J, Hu H, Pan C, Zhou Y, Li L. Scheduling dual-arm cluster tools with multiple wafer types and residency time constraints. IEEE/CAA J Autom Sin 2020;7:776–89. https://doi.org/10.1109/JAS.2020.1003150.

[13] Wang J, Liu C, Zhou MC, Abusorrah A. Virtual cell-based scheduling approach to single-robotic-arm cluster tools subject to wafer residency time constraints. IEEE Trans Autom Sci Eng 2025;22:240–51. https://doi.org/10.1109/TASE.2024.3349380.

[14] Yang F, Li C, Zhen L, Zhang C, Wu N. Scheduling residency time-constrained single-armed cluster tools with two-wafer types via mixed integer programming model. Expert Syst Appl 2025;288:128209. https://doi.org/10.1016/j.eswa.2025.128209.

[15] Kim H, Kim HJ, Lee JH, Lee TE. Scheduling dual-armed cluster tools with cleaning processes. Int J Prod Res 2013;51:3671–87. https://doi.org/10.1080/00207543.2012.758392.

[16] Yu TS, Lee TE. Wafer delay analysis and control of dual-armed cluster tools with chamber cleaning operations. Int J Prod Res 2020;58:434–47. https://doi.org/10.1080/00207543.2019.1593547.

[17] Qiao Y, Lu Y, Li J, Zhang S, Wu N, Liu B. An efficient binary integer programming model for residency time-constrained cluster tools with chamber cleaning requirements. IEEE Trans Autom Sci Eng 2022;19:1757–71. https://doi.org/10.1109/TASE.2021.3122576.

[18] Li C, Yang F, Zhen L. Efficient scheduling approaches to time-constrained single-armed cluster tools with condition-based chamber cleaning operations. Int J Prod Res 2022;60:3555–68. https://doi.org/10.1080/00207543.2021.1926568.

[19] Zhu Q, Yuan J, Wang G, Hou Y. Scheduling a single-arm two-cluster tool with a process module failure subject to wafer residency time constraints. IEEE Trans Autom Sci Eng 2025;22:667–79. https://doi.org/10.1109/TASE.2023.3275703.

[20] Kim D, Kim HJ. Noncyclic scheduling of cluster tools using deep reinforcement learning. IEEE Trans Autom Sci Eng 2025;22:24009–23. https://doi.org/10.1109/TASE.2025.3632534.

[21] Lee JH, Zhao C, Li J, Papadopoulos CT. Analysis, design, and control of Bernoulli production lines with waiting time constraints. J Manuf Syst 2018;46:208–20. https://doi.org/10.1016/j.jmsy.2018.01.001.

[22] Weng W, Chen J, Zheng M, Fujimura S. Realtime scheduling heuristics for just-in-time production in large-scale flexible job shops. J Manuf Syst 2022;63:64–77. https://doi.org/10.1016/j.jmsy.2022.01.006.

[23] Sun L, Lyu P, Zhang X, Wu J, Wang Q, Ervin J, et al. Machine learning technologies for semiconductor manufacturing. In: 2024 Conference of Science and Technology for Integrated Circuits (CSTIC). IEEE; 2024, p. 1–7. https://doi.org/10.1109/CSTIC61820.2024.10532076.

[24] Chen CY, Chang SC, Liao DY. Equipment anomaly detection for semiconductor manufacturing by exploiting unsupervised learning from sensory data. Sensors 2020;20:5650. https://doi.org/10.3390/s20195650.

[25] Biebl F, Glawar R, Jalali A, Ansari F, Haslhofer B, de Boer P, et al. A conceptual model to enable prescriptive maintenance for etching equipment in semiconductor manufacturing. Procedia CIRP 2020;88:64–9. https://doi.org/10.1016/j.procir.2020.05.012.

[26] Rahmani D, Heydari M. Robust and stable flow shop scheduling with unexpected arrivals of new jobs and uncertain processing times. J Manuf Syst 2014;33:84–92. https://doi.org/10.1016/j.jmsy.2013.03.004.

[27] Kim HJ, Lee JH. Scheduling cluster tools for concurrent processing: Deep reinforcement learning with adaptive search. IEEE Trans Autom Sci Eng 2025;22:3783–96. https://doi.org/10.1109/TASE.2024.3399818.

[28] Tang Y, Agrawal S, Faenza Y. Reinforcement learning for integer programming: Learning to cut. In: Proceedings of the 37th International Conference on Machine Learning. PMLR; 2020, p. 9367–76. https://proceedings.mlr.press/v119/tang20a.html.

[29] Huang Z, Wang K, Liu F, Zhen HL, Zhang W, Yuan M, et al. Learning to select cuts for efficient mixed-integer programming. Pattern Recognit 2022;123:108353. https://doi.org/10.1016/j.patcog.2021.108353.

[30] Paulus MB, Zarpellon G, Krause A, Charlin L, Maddison C. Learning to cut by looking ahead: Cutting plane selection via imitation learning. In: Proceedings of the 39th International Conference on Machine Learning. PMLR; 2022, p. 17584–600. https://proceedings.mlr.press/v162/paulus22a.html.

[31] Wang Z, Li X, Wang J, Kuang Y, Yuan M, Zeng J, et al. Learning cut selection for mixed-integer linear programming via hierarchical sequence model. In: Proceedings of the Eleventh International Conference on Learning Representations. 2023. https://openreview.net/forum?id=Zob4P9bRNcK.

[32] Achterberg T. SCIP: Solving constraint integer programs. Math Program Comput 2009;1:1–41. https://doi.org/10.1007/s12532-008-0001-1.

[33] Bestuzheva K, Besançon M, Chen WK, Chmiela A, Donkiewicz T, van Doornmalen J, et al. Enabling research through the SCIP optimization suite 8.0. ACM Trans Math Softw 2023;49:22:1–22:21. https://doi.org/10.1145/3585516.

[34] Wang J, Wang Z, Li X, Kuang Y, Shi Z, Zhu F, et al. Learning to Cut via Hierarchical Sequence/Set Model for Efficient Mixed-Integer Programming. [Preprint]. arXiv:2404.12638; 2024. https://arxiv.org/abs/2404.12638.

[35] Vinyals O, Fortunato M, Jaitly N. Pointer networks. In: Advances in Neural Information Processing Systems. 2015, p. 2692–700.

[36] Choo J, Kwon YD, Kim J, Jae J, Hottung A, Tierney K, et al. Simulation-guided beam search for neural combinatorial optimization. In: Advances in Neural Information Processing Systems. 2022. https://doi.org/10.52202/068431-0637.

[37] Nemhauser GL, Wolsey LA, Fisher ML. An analysis of approximations for maximizing submodular set functions—i. Math Program 1978;14:265–94. https://doi.org/10.1007/BF01588971.

[38] Williams RJ. Simple statistical gradient-following algorithms for connectionist reinforcement learning. Mach Learn 1992;8:229–56. https://doi.org/10.1007/BF00992696.

[39] Mnih V, Badia AP, Mirza M, Graves A, Lillicrap TP, Harley T, et al. Asynchronous methods for deep reinforcement learning. In: Proceedings of the 33rd International Conference on Machine Learning. PMLR; 2016, p. 1928–37. https://proceedings.mlr.press/v48/mniha16.html.